Parameterized Verification

Abstract

Parameterized verification is the problem of establishing that a correctness property holds for a family of systems indexed by a natural number n, representing the number of components, for all values of n simultaneously. The problem is undecidable in general, but a wide range of techniques has been developed that achieve decidability or effective automation for practically important classes of systems.

This chapter surveys the main approaches, using a simple mutual exclusion protocol as a running example. Cutoff methods reduce verification to a finite number of instances by exploiting symmetry: a cutoff theorem asserts that correctness up to some bound k implies correctness for all sizes, and dynamic variants such as view abstraction determine the cutoff adaptively during the analysis. Regular model checking represents configurations as words and transition relations as finite-state transducers, reducing reachability to transducer closure; its lightweight variant, monotonic abstraction, uses upward-closed sets and backward reachability to obtain termination via well-quasi-ordering arguments. Abstraction and network invariant methods reduce the parameterized problem to a finite one by replacing the unbounded environment of a process with a single abstract summary. Deductive and proof-based methods construct inductive invariants that hold symbolically for all n, with automation provided by invisible-invariant synthesis and interactive proof tools. SMT-based methods, including array-based backward reachability, IC3/PDR with quantified invariant inference, and decidable fragments of first-order logic, automate invariant discovery using modern satisfiability solvers.

Two application domains illustrate how these techniques extend to harder settings. For concurrent programs under weak memory models, where hardware reordering introduces unbounded per-thread buffers, a dual load-buffer semantics restores the monotonicity required for backward reachability, giving the first decidability result for parameterized TSO verification and a PSPACE-completeness characterization; the richer Intel x86 model (eTSO) requires better quasi-orderings, marking their first use in program verification. For parameterized quantum circuits, where the parameter counts qubits rather than processes and correctness is a relational input-output property, synchronized weighted tree automata provide a symbolic representation of infinite families of quantum states, and a transducer-based composition framework yields the first fully automatic verification of parameterized quantum programs.

1 Introduction

Modern computing systems are increasingly built around the idea of replication: dozens of processors sharing a cache, hundreds of nodes collaborating in a distributed protocol, thousands

of agents coordinating in a multi-agent system, or an arbitrary number of qubits processed by a quantum algorithm scaled to the problem at hand. A fundamental question arises in all of these settings: how do we know that such systems behave correctly, regardless of how many components are involved?

Traditional formal verification techniques, such as model checking, work by exhaustively exploring the states of a system. They are highly effective when the system is fixed and finite, but become problematic as the system scales. When the number of components (processes, nodes, threads, agents, or qubits) is left as a free parameter, the state space is infinite, and classical finite-state methods no longer apply directly. This is the setting that **parameterized verification** addresses: proving or disproving that a correctness property holds for a *family* of systems, indexed by a natural number n representing the number of components, for all values of n simultaneously.

Properties and systems of interest. The correctness properties most commonly studied in parameterized verification are *safety* properties, which assert that something bad never happens (e.g., mutual exclusion: no two processes occupy a critical section at the same time), and *liveness* properties, which assert that something good eventually happens (e.g., every request is eventually served). Fairness conditions on scheduling, specifying how components take turns, are also of interest, particularly in combination with liveness. In the quantum setting, the relevant correctness notion is different in character: one typically asks for *relational* or *functional* properties, asserting that a parameterized circuit correctly maps every input state to the expected output state, for all input sizes simultaneously.

The range of systems to which parameterized verification applies is broad. Classic benchmarks include *mutual exclusion protocols* such as Burns', Peterson's, Dekker's, and Szymanski's algorithms, where the goal is to show that no two of an arbitrary number of processes can simultaneously enter their critical sections. *Cache coherence protocols*, such as the German and MESI protocols used in multiprocessor hardware, must maintain consistency of shared data across an unbounded number of caches. *Token-passing rings* pass a single token around a linear or circular array of processes, and safety amounts to showing that at most one process holds the token at any time. *Distributed consensus protocols*, such as variants of Paxos and leader election algorithms, must reach agreement among an arbitrary number of participants despite failures. Beyond these classic examples, parameterized verification applies to broadcast communication networks, producer-consumer systems, concurrent programs running on modern multiprocessor hardware under *weak memory models*, and, most recently, parameterized quantum circuits, where the number of qubits scales with the problem instance.

Fundamental difficulty. The challenge of parameterized verification is fundamental. In general, it is undecidable [AK86]: there is no algorithm that, given an arbitrary parameterized sys-

tem and property, correctly determines whether the property holds for all system sizes. This result follows from the ability to simulate the computation of a Turing machine using parameterized processes, and it holds even for very restricted architectures, such as rings of identical finite-state processes [Suz88]. The undecidability of the problem has not, however, prevented significant progress. Instead, it has motivated a wide range of techniques, each identifying meaningful and practically important subclasses of systems for which verification *is* tractable, and developing algorithms, abstractions, and proof methods tailored to those subclasses.

A unifying theme across many of the approaches covered here is the need for a *finite symbolic representation* of infinite families of system states. In the classical setting, regular languages and upward-closed sets play this role; in the quantum setting, *synchronized weighted tree automata* serve an analogous purpose. Another recurring theme is *monotonicity*: the observation that adding more components to a system can only enable more behavior, not less, is exploited in cutoff arguments, backward reachability algorithms, and well-structured transition system frameworks alike. A third theme is the tension between automation and expressiveness: fully automatic methods necessarily restrict the class of systems or properties they handle, while deductive and proof-based methods gain generality at the cost of requiring more human guidance.

Overview of approaches. This chapter covers the main lines of work in parameterized verification, organized around the following themes.

Parameterized systems (Section 2) are introduced informally, together with the verification problem, a running example of a simple mutual exclusion protocol that we revisit in later sections, and the basic notions of well-quasi-orderings and monotonic transition systems on which several of the later techniques rely.

Early foundations (Section 3) trace the origins of the field to the first formulations of the parameterized model-checking problem: a control process together with an unbounded population of identical user processes communicating by rendez-vous, for which reasoning about the whole family reduces to a finite analysis [GS92]; automata used as inductive invariants for linear networks of arbitrary size [SG99]; and symmetry reduction for populations of identical processes [ES96, GS97].

Cutoff methods (Section 4) exploit the symmetry and homogeneity of parameterized systems to reduce verification to a finite number of instances. A *cutoff theorem* asserts that if a property holds for all system sizes up to some bound k, it holds for all sizes. Static cutoff theorems, due to Emerson, Namjoshi, and Kahlon, apply to token-passing rings, guarded protocols, and cache coherence protocols [EN95, EK00, EK03]. *Dynamic cutoff methods*, exemplified by *view abstraction* [AHH13, AHH16], determine the cutoff adaptively during verification.

Regular model checking (Section 5) takes a symbolic approach, representing configurations as words over a finite alphabet and

sets of configurations as regular languages [KMM+01, WB98, BJNT00]. Transitions are encoded as length-preserving finite-state transducers, and reachability analysis reduces to computing the transitive closure of a transducer. A lightweight variant, *monotonic abstraction*, replaces full regular-language representations with upward-closed sets, enabling efficient backward reachability analysis that terminates by a well-quasi-ordering argument [ADBHR07, Abd12].

Abstraction and network invariants (Section 6) reduce the parameterized problem to a finite one by constructing a small abstract system that over-approximates all instances. The *network invariant* method represents the collective behavior of all processes except one by a single abstract process [WL89, KM89, KPSZ02]. Counter abstraction and environment abstraction summarize the population by bounded counters or by the view of a single reference process [PXZ02, CTV06].

Deductive and proof-based methods (Section 7) approach parameterized verification by constructing formal proofs of inductive invariants that hold for all system sizes simultaneously, building on the work of Owicki and Gries [OG76] and Lamport [Lam94]. Invisible invariants and related techniques reduce the burden on the user by generating candidate invariants automatically from small instances [PRZ01].

SMT-based methods (Section 8) use modern Satisfiability Modulo Theories solvers to automate the search for inductive invariants. Array-based system encodings [GR10a, GR10b, CGK+12] support backward reachability with quantified formulas. The IC3/PDR algorithm [Bra11], originally designed for finite-state hardware verification, has been extended to the parameterized setting through quantified invariant inference [KBI+17, GSM16]. Tools such as Ivy [PMP+16] let users specify systems in decidable fragments of first-order logic and derive invariants that certify safety for all n.

Applications to weak memory models (Section 9) require classical techniques to handle the unbounded buffers that arise from hardware memory reordering. The *dual semantics* for TSO makes the system monotone and amenable to backward reachability, yielding the first decidability result for parameterized TSO verification [AABN16, AABN18]. The precise complexity of this problem was subsequently shown to be PSPACE-complete [AAR20]. The richer Intel x86 model (eTSO) requires the machinery of *better quasi-orderings* [AAB+26].

Parameterized verification of quantum circuits (Section 10) is a recent direction, in which the parameter n counts qubits rather than processes, and the correctness criterion is functional rather than safety-oriented. A new automata model, *synchronized weighted tree automata*, provides a symbolic representation of infinite families of quantum states, and a transducer-based framework handles parameterized gate semantics and circuit composition [ACH+26].

Scope and organization. Sections 3–10 discuss the underlying ideas, key results, representative algorithms, and important

benchmarks of each approach. Section 11 discusses industrial applications of parameterized verification. Section 12 situates this chapter within the broader context of the book, drawing connections to related topics covered in other chapters. Section 13 concludes with open problems. Throughout, we aim for an accessible presentation with minimal formalism, providing intuition and representative examples in preference to full technical detail. Pointers to primary sources are given for readers who wish to explore individual topics further.

Several surveys complement this chapter. The chapter [AST18] gives a technical account of cutoffs, regular model checking, monotonic abstraction, and environment abstraction. The monograph [BJK+15] systematically maps the boundary between decidable and undecidable classes of parameterized systems according to their communication primitives and topologies, and the overview [Esp14] focuses on the computational complexity of parameterized verification for populations of anonymous processes. Earlier surveys cover abstraction-based methods [ZP04], regular model checking [AJNS04, Abd12, Abd21], and the theory of well-quasi-ordered transition systems [Abd10].

2 Parameterized Systems

This section fixes the setting for the rest of the chapter. We describe parameterized systems and the verification problem informally, introduce a running example, and recall the basic notions of well-quasi-orderings and monotonic transition systems, which underlie several of the techniques presented later. We keep the presentation informal; precise definitions for the different classes of systems can be found in the cited works [AST18, BJK+15, Abd10].

2.1 Process Templates and Configurations

A parameterized system is defined by a *process template*, a finite-state machine that describes the behavior of a single process. The template has a finite set Q of *local states*, a set of initial local states, and a set of transitions. For each natural number n, the *instance* of size n consists of n copies of the template running in parallel, where the copies are identified by the indices $1, \ldots, n$. Some systems also contain a distinguished *control process*, defined by a separate template, that runs in parallel with the copies.

A *configuration* of the instance of size n records the local state of each process. We represent it as a word $q_1 q_2 \cdots q_n$ of length n over the alphabet Q, where q_i is the local state of process i. When the system also has shared variables, the configuration includes their values. For each fixed n, the instance is a finite transition system. The parameterized system is the infinite family of all instances, and its set of configurations, taken over all n, is infinite.

The *topology* of a system determines how processes are arranged and which processes can observe each other. In a *linear* topology, the processes form an array, and a process can distinguish the processes to its left from those to its right. In a *ring*, the last process is connected back to the first. In a *clique*, all pro-

cesses are indistinguishable; the order of the letters in a configuration is then irrelevant, and a configuration can be represented by a multiset of local states, or equivalently by a vector that counts, for each local state, how many processes occupy it. Trees and more general graphs also occur, for instance in hierarchical protocols and in ad hoc networks.

Processes interact through a small number of communication primitives. A *local* transition changes the state of a single process without involving the others. In a *rendez-vous*, two processes synchronize and change their states simultaneously. In a *broadcast*, one process sends a message and all other processes react to it in the same step. Processes may also read and write *shared variables* with finite domains. Finally, a transition may be guarded by a *global condition* that refers to the states of the other processes. The condition is *universal* if it requires all other processes (or all processes to the left, or to the right) to be in given local states, and *existential* if it requires at least one of them to be. Global conditions are a convenient way of modeling protocols in which a process inspects the states of its peers before taking a step.

2.2 The Verification Problem

Most of this chapter is concerned with safety properties. A safety property is given by a set *Bad* of *bad* configurations, which is defined independently of the size of the instance. For instance, for mutual exclusion, *Bad* is the set of configurations, of any length, in which at least two processes are in their critical sections. We write *Init* for the set of initial configurations of all instances. The *parameterized reachability problem* asks whether there is a size n and a run of the instance of size n that starts from a configuration in *Init* and reaches a configuration in *Bad*. The system is *safe* if no such n and run exist, that is, if the union of the reachable sets of all instances is disjoint from *Bad*.

More general properties are often expressed in *indexed* temporal logics [BCG89, EN95, GS92], whose formulas quantify over process indices. A formula of the form "for all distinct indices i and j, $\varphi(i,j)$" states that the temporal property φ holds for every pair of processes. Liveness properties, such as the requirement that every process that requests access to its critical section eventually obtains it, are typically checked under fairness assumptions on the scheduler.

As mentioned in Section 1, parameterized verification is undecidable in general [AK86]. The proof encodes the tape of a Turing machine in the configuration of a linear array of processes, one process per tape cell, with the head stored in the local state of one of the processes. Since the number of processes is unbounded, every finite portion of the tape can be represented, and the halting problem reduces to the parameterized reachability problem. The same idea works for rings of identical processes [Suz88]. Each technique in this chapter can be seen as a response to this result: it either identifies a class of systems for which the problem is decidable, or accepts incompleteness in exchange for generality.

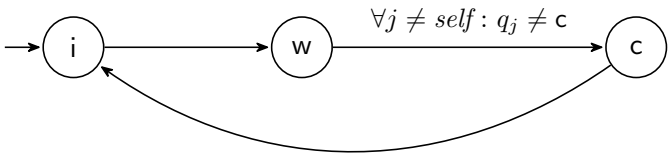


Figure 1: The process template of the running example. The transition from w to c is guarded by a universal condition: it is enabled only if no other process is in c.

2.3 A Running Example

We use the following simple mutual exclusion protocol as a running example. Each process has three local states: i (idle), w (waiting), and c (in the critical section). Initially, all processes are in i. A process may move from i to w at any time. A process in w may enter the critical section, moving to c, if no other process is in c; this is a universal global condition. A process in c may return to i at any time. The template is shown in Figure 1.

Configurations are words over the alphabet $\{\mathsf{i}, \mathsf{w}, \mathsf{c}\}$, and the initial configurations are the words i^n, for $n \geq 1$. The bad configurations are the words that contain at least two occurrences of c. For example, the instance of size three can reach wiw from iii in two steps, and then move to ciw and to iiw, but it can never reach a configuration such as cic, since the second process in w is blocked as long as the first one is in c.

The protocol is safe for every n, and for a human reader this is obvious: the guard is checked and the state is changed in a single atomic step, so no process can enter the critical section while another one is there. The example is therefore not interesting as a verification challenge, but it is small enough to show how each technique handles the unbounded number of processes. Realistic protocols, such as Burns' or Szymanski's algorithms, replace the atomic universal condition by a loop in which a process inspects the other processes one at a time. Their correctness then depends on subtle interactions between interleaved inspections, and verifying them is considerably harder [ABHDR08, AHH16].

2.4 Well-Quasi-Orderings and Monotonic Systems

Several techniques in this chapter rely on orderings on configurations that compare instances of different sizes. A *quasi-ordering* on a set is a reflexive and transitive relation $\sqsubseteq$. We say that $\sqsubseteq$ is a *well-quasi-ordering* (WQO) if every infinite sequence $a_0, a_1, a_2, \ldots$ contains two elements a_i and a_j with $i < j$ and $a_i \sqsubseteq a_j$. For words, we use the *subword ordering*. We write $u \preceq v$ if u can be obtained from v by deleting letters, that is, if u occurs in v as a not-necessarily-contiguous subsequence. For example, $\mathsf{wc} \preceq \mathsf{wiic}$. Two classical results provide the WQOs used most often in parameterized verification. By Dickson's lemma, vectors of natural numbers of a fixed dimension, ordered componentwise, form a WQO. By Higman's lemma [Hig52], the words over a finite alphabet form a WQO under the subword ordering. Higman's lemma extends to words over any well-quasi-ordered alphabet, which makes it possible to combine orderings in a mod-

ular way.

A set U of configurations is *upward-closed* if it contains, together with every element a, every element b with $a \sqsubseteq b$. Symmetrically, U is *downward-closed* if it contains, together with every element b, every element a with $a \sqsubseteq b$. We write $\uparrow a$ for the set of elements larger than or equal to a. In the running example, the set of bad configurations is upward-closed under the subword ordering: it is the set $\uparrow(\mathsf{cc})$ of words that contain cc as a subword. Upward-closed sets play a central role for two reasons. First, under a WQO, every upward-closed set is determined by its finitely many minimal elements, which gives a finite representation of an infinite set. Second, every increasing sequence $U_0 \subseteq U_1 \subseteq \cdots$ of upward-closed sets eventually stabilizes.

We say that a transition system is *monotonic* with respect to an ordering $\sqsubseteq$ on its configurations if larger configurations can simulate smaller ones: whenever a can move to a' in one step and $a \sqsubseteq b$, the configuration b can move in one step to a configuration b' with $a' \sqsubseteq b'$. A *well-structured transition system* is a transition system that is monotonic with respect to a WQO [AČJT96,AČJT00,FS01]. In a well-structured system, the set of configurations from which an upward-closed set U can be reached is itself upward-closed. This set can be computed by *backward reachability*: one starts from $U_0 = U$ and computes $U_{i+1} = U_i \cup \mathit{Pre}(U_i)$, where $\mathit{Pre}(U_i)$ is the set of immediate predecessors of U_i. Each U_i is upward-closed and is represented by its minimal elements, and the sequence stabilizes after finitely many steps. Safety then amounts to checking that no initial configuration belongs to the limit. The algorithm requires that the minimal elements of $\mathit{Pre}(U)$ can be computed from those of U, which is the case for most models considered in this chapter.

Many parameterized systems are monotonic under the subword ordering. Local transitions, rendez-vous, broadcast, and existential global conditions do not depend on the number of other processes in a way that could disable them, so adding processes preserves the ability to perform them. Universal global conditions are different. In the running example, the configuration w can move to c, but the larger configuration wc cannot move its waiting process, because the added process is in the critical section. The running example is therefore not monotonic, and neither are most mutual exclusion protocols. Sections 4 and 5 describe two ways of dealing with this: view abstraction, which does not require monotonicity, and monotonic abstraction, which restores it by over-approximation.

For systems with a clique topology, a configuration can be represented by a vector of counters, one for each local state. This *counting abstraction* is exact when processes are anonymous, and it turns the system into a Petri net, or equivalently a vector addition system, in which each transition adds a fixed vector to the current counter values [GS92, Esp98]. Petri nets are monotonic with respect to the componentwise ordering, which is a WQO by Dickson's lemma, so their coverability problem, and hence the parameterized reachability problem for such systems, is decidable. A universal global condition corresponds to a test that a counter

is zero, which Petri nets cannot express. This is another view of why universal conditions are the main source of difficulty in the analysis of parameterized systems.

3 Early Foundations

The questions studied in this chapter were first formulated in a body of work from the late 1980s and 1990s that established both the model and the basic techniques of parameterized verification. This section reviews that work and some of its later developments.

Correspondence between instances. While the undecidability results of Section 2 showed early on that a general algorithm cannot exist [AK86, Suz88], Browne, Clarke, and Grumberg [BCG89] proposed a way of transferring verification results from small instances to large ones. They introduced an indexed version of the temporal logic CTL* without the next-time operator, together with a notion of *correspondence* between systems of different sizes that preserves the formulas of this logic. Once a correspondence between a small instance and all larger instances has been established, model checking the small instance suffices. Establishing the correspondence requires, in general, an argument by the user, but the approach anticipates the cutoff theorems of Section 4, in which the transfer from small to large instances is guaranteed by the structure of the system.

Systems with many processes. A foundational formulation considers a system composed of a single control process together with an unbounded collection of user processes sharing a common definition, communicating through rendez-vous synchronization in the style of CCS [GS92]. The central contribution was to show that reasoning about the entire infinite family can be reduced to a finite analysis: decision procedures were given for checking whether every process satisfies a given indexed temporal specification. When all processes are identical, the procedure runs in time polynomial in the size of the process template; in the presence of a control process, it runs in doubly exponential time. The reachability questions arising in the analysis were connected to the theory of Petri nets and vector addition systems, through the counting abstraction described in Section 2.4: since the user processes are anonymous, it suffices to record how many of them are in each local state. The question of whether some user process can reach a given local state becomes the *coverability* problem for Petri nets, which is decidable and, by the results of Lipton and Rackoff, ExpSpace-complete [Rac78, Esp98]. This early demarcation of the boundary between decidable and undecidable cases, and of the algebraic structures underlying it, shaped much of the work that followed.

Induction over the number of processes. A second early line of work used induction on the number of processes. Kurshan and McMillan [KM89], and independently Wolper and Lov-

infosse [WL89], observed that if a process I is found such that I *abstracts* a single copy of the template, in a sense that preserves the properties of interest, and the composition of I with one more copy is again abstracted by I, then I abstracts the composition of any number of copies. The process I is then a *network invariant*, and verifying the composition of I with a fixed environment is sufficient for all sizes. We return to this idea in Section 6. Clarke, Grumberg, and Jha [CGJ97] generalized the setting to families of networks generated by *network grammars*, using abstraction to make the search for invariants effective.

A complementary line of work targeted linear networks of arbitrary size and introduced automata as inductive invariants [SG99]. A correctness property is itself specified by an automaton, from which an invariant automaton is synthesized; an induction on the number of processes then establishes that the property is preserved as the network grows. The method was shown to be complete for safety properties. It directly anticipates both the network-invariant approach of Section 6 and the regular model checking framework of Section 5, in which sets of global configurations of unbounded width are represented symbolically by finite automata.

Symmetry. A further theme running through this early work is the exploitation of the symmetry inherent in a population of identical processes. Because the identity of an individual process is immaterial to the global behavior of the system, verification can be carried out over equivalence classes of configurations rather than over configurations themselves [ES96, CEFJ96, ID96]. Two configurations are equivalent if one is obtained from the other by permuting the process indices, and the quotient of the state space under this equivalence can be exponentially smaller than the original. Symmetry reduction applies to each instance separately, so it does not by itself solve the parameterized problem, but it explains why configurations of symmetric systems can be represented by multisets, and it is the basis of the counting abstraction above. Combining symmetry with fairness leads to on-the-fly model checking procedures that keep the explored state space manageable [GS97].

Broadcast protocols and cache coherence. Esparza, Finkel, and Mayr [EFM99] studied *broadcast protocols*, in which a process can send a message that is received by all other processes in the same step. Broadcast protocols are an extension of Petri nets, in which a transition can move all tokens from one place to another. They are well-structured, so safety properties expressed by upward-closed sets can be checked by the backward reachability algorithm of Section 2.4, while liveness properties become undecidable. Delzanno [Del00] applied this framework to cache coherence protocols. Using the counting abstraction and symbolic representations based on linear arithmetic constraints, he verified several coherence protocols automatically for an arbitrary number of caches. This was one of the first demonstrations that parameterized verification could handle protocols

of practical relevance.

Later developments. These ideas have been extended in many directions. Parameterized systems with pairwise rendezvous have been analyzed on a range of topologies [AKR+18]. In *asynchronous shared-memory systems*, an unbounded number of anonymous processes communicate through shared registers without locks [EGM16]. In *population protocols*, anonymous agents interact in pairs to compute a predicate of their initial states [AAD+06], and verifying that a protocol computes the intended predicate for every population size is a parameterized problem that can be reduced to problems on Petri nets [EGLM17]. In *ad hoc networks*, processes communicate by broadcast to their neighbors in an unknown graph; the problem is undecidable in general, and becomes decidable under restrictions on the possible topologies [DSZ10]. A recurring observation in this line of work is that, when processes are anonymous and any behavior of one process can be replicated by others, the complexity of parameterized verification is often much lower than that of Petri net coverability [Esp14].

Taken together, these strands form the conceptual foundation on which the remainder of this chapter builds: finite reductions for synchronizing populations, induction over the number of processes, automata-based invariants for linear networks, symmetry reduction for replicated components, and well-structured models of broadcast.

4 Cutoff Methods and Dynamic Cutoffs

A central idea in parameterized verification is the *cutoff theorem*. The observation behind it is that, for certain classes of systems, one does not need to verify infinitely many instances. Instead, it suffices to verify systems up to some finite bound, the cutoff, and correctness for all larger instances follows automatically [EN95,EK00].

Intuitively, this works because processes in many symmetric systems are interchangeable. If the behavior of a system with n processes can be faithfully represented by a system with at most k processes (the cutoff), then verifying the k-process system is enough. Cutoff results have been established for important classes such as token-passing rings, guarded protocols, and systems where processes communicate in restricted ways. This section first presents such *static* cutoffs, and then *dynamic* cutoffs, which are discovered during verification.

4.1 Static Cutoffs

We say that a number k is a *cutoff* for a class of parameterized systems and properties if, for every system and every property in the class, the property holds for all instances of the system whenever it holds for all instances of size at most k. A cutoff theorem reduces parameterized verification to a finite number of finite-state model checking problems, which can be solved with standard tools. The cutoff typically depends on the class of systems, on the number of process indices mentioned in the property,

and sometimes on the size of the process template.

Token rings. Emerson and Namjoshi [EN95] studied rings of identical processes in which a single token, carrying no data, is passed from each process to its neighbor. They showed that properties in indexed temporal logic that refer to a fixed number of processes have a cutoff that is a small constant, independent of the process template. The reason is that a process in a token ring observes the rest of the ring only through the arrival and departure of the token, and from the point of view of a few processes, a large ring behaves like a small one in which the remaining processes have been merged. The result was later extended from rings to other network topologies by decomposing a network into small fragments [CTTV04], and the existence of cutoffs for token-passing systems has been characterized in terms of the topology and the specification logic [AJKR14].

Guarded protocols. Emerson and Kahlon [EK00] considered systems of identical processes whose transitions are guarded by conditions on the other processes. A guard is *conjunctive* if it requires all other processes to be in a given set of local states, and *disjunctive* if it requires at least one other process to be in such a set. They established cutoffs that grow linearly with the number of local states of the template. For disjunctive guards, the argument is a simulation: in a run of a large instance, each guard needs only one witness, and for each local state that occurs in the run it suffices to keep one process that reaches this state and then stays there, so that it can serve as a witness whenever needed. A small instance with these extra processes can reproduce the behavior of the processes mentioned in the property. For conjunctive guards, the argument uses projection instead: removing processes can only make a conjunctive guard easier to satisfy, so the projection of a run of a large instance onto a subset of its processes is a run of a smaller instance. For safety properties this argument is direct, while liveness properties require more care, since the projection of a fair run need not be fair, and deadlocks of the large instance need not appear in the small one. The cutoffs were later sharpened, and extended to fairness and deadlock detection [AJK16], and specialized cutoff methods were developed for cache coherence protocols [EK03].

The running example of Section 2.3 is a guarded protocol: the transition from w to c has the conjunctive guard that all other processes are in i or w. If two processes are in c at the same time in a run of some instance, then the projection of the run onto these two processes is a run of the instance of size two that violates mutual exclusion. Checking the instance of size two, which has only nine configurations, is therefore enough to establish mutual exclusion for all sizes.

Limitations. Static cutoff theorems are attractive because they reduce the problem to standard model checking. However, the bound k is fixed ahead of time and must be established before verification begins. This can be limiting: the cutoff may

be large, difficult to compute, or may not exist at all for systems with richer interaction patterns or more complex topologies. Moreover, each cutoff theorem covers a specific class of systems, and a protocol that falls slightly outside the class, for instance because it combines conjunctive guards with shared variables, is not covered.

4.2 Dynamic Cutoffs and View Abstraction

Dynamic cutoff methods address these limitations by allowing the abstraction boundary to be determined adaptively, during the verification process itself, rather than fixed in advance. A well-developed technique in this family is **view abstraction** [AHH13, AHH16]. Rather than committing to a single small representative system upfront, the method reasons about what a process can *observe* about the rest of the system, and uses those local observations to construct an on-the-fly abstraction.

The starting point is to model the global state of an n-process system as a configuration: a word of local process states, as in Section 2. A *view* of size k is a subword of length k of a configuration, that is, a projection of the configuration onto a subset of k processes that preserves their order. Instead of tracking full configurations (whose length varies with n), the method tracks the set of all views of size up to k that are reachable across *any* instance of the system.

For a given precision parameter $k \in \mathbb{N}$, the framework runs two procedures in parallel. The first is an *under-approximation*: it performs concrete reachability analysis on instances of fixed size up to k, and reports a bug if a bad configuration is found. The second is an *over-approximation* based on abstract interpretation. We write α_k for the abstraction function, which maps a configuration to its set of subwords of size up to k. We write γ_k for the corresponding concretization function, which maps a set of views to the set of all configurations whose subwords of size up to k all belong to the given set of views. A fixpoint V_k is then computed over views using an abstract post operator. The key soundness property is that the set of reachable configurations R satisfies $R \subseteq \gamma_k(V_k)$ for all k, and the approximation becomes tighter as k grows: $\gamma_{k+1}(V_{k+1}) \subseteq \gamma_k(V_k)$.

An important observation for the implementation is that the abstract post operator can be computed using only concrete configurations of size up to $k{+}1$, rather than the full (infinite) concretization. This is because the witnesses required to enable a global transition consist of a single extra process; accommodating one additional process beyond the view size is always sufficient. This makes the fixpoint computation entirely finite and automatic.

If the over-approximation establishes $\gamma_k(V_k) \cap \mathit{Bad} = \emptyset$, the system is declared safe. If the under-approximation finds a reachable bad configuration, the system is declared unsafe. In either case, the current k is a *dynamic cutoff*, a point discovered during verification beyond which the search need not continue. If neither procedure is conclusive, the precision is increased by incrementing k and the analysis is repeated.

The running example. Consider the running example with $k = 1$. The views of size one are single local states, and all three local states i, w, and c are reachable. The concretization of this set of views contains every configuration, including cc, so the over-approximation cannot prove mutual exclusion, and the under-approximation, which only explores the instance of size one, finds no bug. The precision is therefore increased to $k = 2$.

With $k = 2$, the analysis starts from the views of the initial configurations, namely i and ii. The abstract post operator considers the configurations of size at most three whose views are all in the current set, applies the transitions, and collects the views of the results. In the first rounds, the transition from i to w produces the views w, iw, wi, and ww, and the transition from w to c then produces c, ic, ci, wc, and cw. The view cc is never produced: to produce it, a process in w would have to move to c in a configuration of size at most three that already contains a process in c, and the guard forbids this. The fixpoint V_2 therefore consists of all words of length at most two except cc. Its concretization is the set of configurations with at most one process in c, which is disjoint from the bad configurations. The method thus proves mutual exclusion for all sizes, with dynamic cutoff 2.

Termination and applications. The procedure is guaranteed to terminate whenever the set of reachable configurations is downward-closed with respect to the subword ordering, as defined in Section 2.4. It also terminates for *monotonic systems* in the sense of Section 2.4 when the bad configurations form an upward-closed set. In that case, safety depends only on the downward closure of the reachable set, which is closed under the transitions and is represented exactly by views of bounded size. Monotonic systems cover a broad and well-studied class, including lossy channel systems, Petri nets (where the method solves the coverability problem), and timed Petri nets. The running example is not monotonic, but its reachable set, the set of configurations with at most one process in c, is downward-closed, so the procedure terminates as well. For Burns' mutual exclusion protocol, the method likewise terminates at $k = 2$: already with two-process views it can be shown that no bad configuration is reachable for any system size.

View abstraction has been successfully applied to a range of protocols, including mutual exclusion algorithms such as the fine-grained version of Szymanski's protocol, a benchmark that, to the knowledge of the authors of [AHH16], had not been verified automatically by other methods, as well as cache coherence protocols and distributed systems with complex process interactions. The framework extends naturally to other process topologies such as multisets, rings, and trees, and to settings where global transitions are not assumed to be atomic [AHH13,AHH16]. Further refinements of the abstraction, which take into account the context in which a process takes a step, have been developed in [AHH14].

Other dynamic approaches pursue the same goal by different means. Kaiser, Kroening, and Wahl [KKW10] consider programs in which an unbounded number of threads execute the same code

over shared variables. They explore instances of increasing size and detect, during the exploration, a point beyond which additional threads cannot reach new local states. Like view abstraction, the method determines the cutoff from the program at hand instead of from a general theorem.

Together, static cutoff theorems and dynamic approaches like view abstraction form a complementary toolkit: static cutoffs offer clean theoretical guarantees for specific protocol families, while dynamic methods provide greater flexibility and automation for protocols that fall outside those families. The idea of reasoning about a parameterized system through its small instances also reappears in the invisible invariants method of Section 7 and in the small-model properties of the decidable logics used by SMT-based tools in Section 8.

5 Regular Model Checking

Regular Model Checking (RMC) is a symbolic framework for verifying parameterized and infinite-state systems by representing both sets of configurations and transition relations using the theory of regular languages and finite automata [KMM$^+$01, WB98, BJNT00]. The key idea is that regular sets serve as an efficient symbolic representation of infinite state spaces, playing a role analogous to that of Binary Decision Diagrams (BDDs) in symbolic model checking of finite-state systems [AJNS04, Abd12, Abd21].

5.1 The Framework

In its basic form, RMC represents a system as follows. A *configuration* is a finite word over a fixed alphabet Σ, where each symbol encodes the local state of one component (a process, a stack element, a channel message, etc.). Sets of configurations, including the initial set and the set of bad configurations, are represented as regular languages over Σ. The transition relation is encoded as a *length-preserving finite-state transducer* T over pairs $(a, a') \in \Sigma \times \Sigma$: the transducer reads two words of the same length in parallel, letter by letter, and accepts the pair (w, w') if and only if the system can move from configuration w to configuration w' in one step.

This simple setup already captures a wide variety of models. Parameterized systems consisting of an arbitrary number of finite-state processes in a linear or ring topology can be modeled by letting each position in the word represent one process. Systems operating on unbounded queues, stacks, or integer variables can similarly be encoded by letting word positions represent data structure contents. The length-preserving constraint on the transducer is not a restriction for safety verification: for parameterized systems it simply means that the number of processes is fixed at the start of a run, while being arbitrary across runs.

A token-passing example. Consider a linear array of processes in which a single token is passed from left to right. Each process is either holding the token or not holding it. We write `t` and `n` for these two local states. The initial configurations are

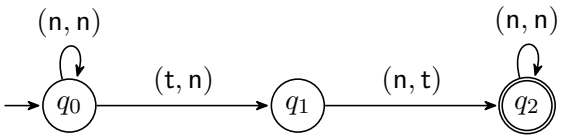


Figure 2: A length-preserving transducer for passing the token one position to the right. Each transition reads a letter of the current configuration together with a letter of the next configuration.

those in which the leftmost process holds the token, that is, the regular language tn^*. In one step, the process holding the token passes it to its right neighbor. Figure 2 shows a transducer T for this transition relation. It copies a prefix of processes without the token, then reads the pair (t, n) for the process that gives the token away, followed by the pair (n, t) for the neighbor that receives it, and finally copies the remaining processes. The safety property states that exactly one process holds the token, so the bad configurations form the regular language of words with no t or with at least two occurrences of t.

Reachability. Let *Init* and *Bad* be the regular sets of initial and bad configurations. We write T^* for the reflexive transitive closure of the transducer relation, and $\mathit{Init} \circ T^*$ for the set of configurations reachable from *Init*. Safety verification reduces to checking whether $(\mathit{Init} \circ T^*) \cap \mathit{Bad} = \emptyset$. Liveness verification reduces to a *repeated reachability* problem: checking whether there exists an infinite computation from *Init* through T that visits a set of accepting configurations F infinitely often.

In the token-passing example, the relation T^k, obtained by composing T with itself k times, moves the token exactly k positions to the right, and each T^k is recognized by a transducer with $k + 2$ states. Iterating T one step at a time therefore produces an infinite sequence of growing transducers and never reaches a fixpoint. The transitive closure T^+, however, is regular: it moves the token any positive number of positions to the right, and it is recognized by the transducer of Figure 2 with an additional loop labeled (n, n) on the state q_1. The reachable set is then $\mathsf{n}^*\mathsf{tn}^*$, which is disjoint from the bad configurations.

5.2 Computing Transitive Closures

The central computational challenge is computing T^+ (or $\mathit{Init} \circ T^*$) effectively. Since the problem is undecidable in general, no method can succeed on all inputs, and naively iterating the transducer, computing $T^1, T^2, T^3, \ldots$, does not converge, as the example shows. The standard approach is to use *acceleration* or *widening* techniques that jump directly to a representation of T^+.

Acceleration by quotienting. Acceleration is typically based on *quotienting*: one finds an equivalence relation $\simeq$ on the states of a finite approximation T^n, merges equivalent states to obtain a quotient transducer $T^n/{\simeq}$, and checks whether the result characterizes T^+. The difficulty is that quotienting generally enlarges the accepted language: $L(T^n) \subseteq L(T^n/{\simeq})$, often strictly. The

quotient is therefore taken on the *history transducer*, a canonical infinite transducer built as the union of all T^n for $n \geq 1$, where each state records the full sequence of T-states visited during product construction. A key theoretical contribution is the identification of equivalence relations on the states of the history transducer whose quotient accepts exactly T^+. Such relations can be derived from forward and backward bisimulations, and in later work from simulations, on these states. When the resulting quotient is finite, it is a transducer for T^+ [AJNd02, AJNd03]. Related constructions iterate transducers by identifying equivalent states with bisimulation-like relations [DLS01].

Extrapolation and abstraction. A different idea is to compute a few iterates $T^1, \ldots, T^k$, detect a regular pattern of growth between consecutive iterates, and *extrapolate* it to a guess of the limit, which is then checked for correctness [BLW03, BT02]. In the token-passing example, the transducer for T^{k+1} has one more state, and one more (n, n) step between the pair that gives the token away and the pair that receives it, than the transducer for T^k. Extrapolation turns this repeated step into the loop on q_1. *Abstract regular model checking* [BHV04] gives up exactness instead: it computes reachable sets by alternately applying the transducer and abstracting the resulting automata, for example by merging states that accept the same words up to a given length. The abstraction guarantees termination, and spurious counterexamples are used to refine it.

Learning regular invariants. For safety verification, it is not necessary to compute the reachable set exactly: it suffices to find a regular set that contains the initial configurations, is closed under the transducer, and is disjoint from the bad configurations. Such a *regular inductive invariant* can be searched for with techniques from automata learning, in which a learner proposes candidate automata and a teacher checks the three conditions and returns counterexamples [HV05, NJ13, CHLR17]. Checking whether a given automaton is an inductive invariant is a decidable problem on automata, so the learning approach replaces the computation of a fixpoint by a sequence of decidable checks.

Extensions. Beyond the linear word model, the framework extends to configurations represented as trees, enabling verification of systems with hierarchical or recursive structure [AJMd02, BT02]. Liveness properties can be handled by extending the framework to infinite words [BLW05], and regular proofs of liveness have been studied for randomized parameterized systems [LR16]. RMC has been successfully applied to token-passing systems, mutual exclusion algorithms, and systems with unbounded communication channels, stacks, and integer variables [AJNS04, Abd12].

5.3 Monotonic Abstraction

While the transducer-based approach is general, it can be computationally expensive and may not scale to larger protocols. An

important line of work within the RMC framework addresses this by replacing the full power of regular languages with a lighter symbolic representation based on *upward-closed sets*, in a technique known as *monotonic abstraction* [ADBHR07, Abd12]. The approach targets parameterized systems with a linear process topology and global transitions (both universal and existential). Configurations are again words over the process state alphabet, and the subword ordering $\preceq$ of Section 2.4 plays a central role. Bad configurations are characterized by upward-closed sets with respect to this ordering: if a configuration is bad, then any larger configuration (with more processes) is also bad. Such sets have a finite symbolic representation in terms of their minimal elements, making them far more tractable than arbitrary regular sets.

The core idea is to replace the concrete transition relation $\rightarrow$ with an abstract relation $\rightarrow_A$ that over-approximates $\rightarrow$ and is monotone with respect to $\preceq$. Local and existential transitions are already monotone and require no approximation. Universal transitions, where a process may move only if all processes in its context satisfy some condition, are *not* monotone, since adding more processes to a configuration may violate the universal condition and disable the transition. The abstraction handles this by allowing a configuration to silently drop processes that obstruct the transition, effectively making the universal condition vacuously satisfiable. Since $\rightarrow_A$ is an over-approximation, any safety property that holds in the abstract model is guaranteed to hold in the concrete one.

Safety verification then proceeds by *backward reachability analysis*, as in Section 2.4. Starting from the upward-closed set Bad of bad configurations, one computes the sequence $U_0 = Bad$, $U_{i+1} = U_i \cup Pre_A(U_i)$, where Pre_A is the predecessor operator under $\rightarrow_A$. Because $\rightarrow_A$ is monotone, each U_i remains upward-closed and can be represented by a finite set of generators. Termination is guaranteed by Higman's lemma: the subword ordering $\preceq$ is a well-quasi-ordering on words, so every ascending chain of upward-closed sets stabilizes after finitely many steps. The algorithm terminates when no new generators (up to subsumption) are produced; if none of the generators represents an initial configuration, the system is safe.

The running example. In the running example, the abstract relation allows a process in w to move to c in any configuration, after deleting the processes that are in c. For instance, $\mathsf{wc} \rightarrow_A \mathsf{c}$. Backward reachability starts from the single generator cc and computes the configurations from which one abstract step leads to a configuration with at least two processes in c. The steps from i to w and from c to i do not create processes in c, so a configuration that reaches $\uparrow(\mathsf{cc})$ by one of these steps already has at least two processes in c. The abstract step from w to c deletes all other processes in c, so it always results in a configuration with exactly one process in c. Hence Pre_A adds no new generators, the fixpoint is $\uparrow(\mathsf{cc})$, and since no initial configuration i^n contains cc, the protocol is safe. In this example the abstraction loses no precision with respect to the property. For protocols in which

a process inspects the others one at a time, such as Burns' and Szymanski's algorithms, the same approach applies to the non-atomic versions of the global conditions [ABHDR08].

Extensions and refinement. Monotonic abstraction has been extended to processes with numerical variables whose values are compared using gap-order constraints [ADR09], to tree topologies, broadcast communication, and binary rendez-vous, and to systems running under weak memory models (Section 9). Since the abstraction may introduce spurious counterexamples, it has been combined with a counterexample-guided refinement loop that restricts the deletions allowed in the abstract transitions [ACD+10]. The connection between monotonic abstraction and well-structured transition systems provides general decidability results for the abstract systems, for classes of systems that extend well beyond the purely parameterized setting [Abd10].

6 Abstraction and Network Invariants

Abstraction-based methods address the parameterized problem by reducing it to a finite, non-parameterized one. The idea is to construct an abstract system, typically a small, fixed-size system, that *over-approximates* the behavior of the full parameterized family. If a safety property holds in the abstract system, it holds in all concrete instances [WL89,PXZ02,ZP04]. The methods in this section differ in what the abstract system describes: the environment of a single process, the number of processes in each local state, or the relationship between a reference process and the others.

6.1 Network Invariants

A particularly influential instantiation of this idea is the *network invariant* method [WL89, KM89], further developed in [KPSZ02,LHR97]. Consider a system consisting of n copies of a process template running in parallel. A network invariant is a single process I (or small composition of processes) that captures the collective behavior of all $n-1$ processes that a given process interacts with. Verifying the property then reduces to checking the two-process composition of one concrete process with the invariant I, regardless of n. If this finite system satisfies the property, and I is a valid over-approximation of the environment, then all n-process instances satisfy it too.

Validity of the invariant is established by induction, as described in Section 3: one checks that I over-approximates a single copy of the template, and that the composition of I with one more copy is again over-approximated by I. Both checks are finite-state problems. The notion of over-approximation is a preorder on processes, such as trace inclusion or simulation, that is preserved by parallel composition and that preserves the properties of interest.

As an illustration, consider the running example from the point of view of one process. The other processes matter to it only through the question of whether one of them is in c. A candidate invariant I has two states, *free* and *busy*. It can move from free

to busy only when the process it is composed with is not in c, and it can move from busy to free at any time. The composition of one process with I satisfies mutual exclusion, since the process enters c only when I is free, and I becomes busy only when the process is not in c. The composition of I with one more process, observed through the same question, behaves again like I, which establishes validity.

Finding the right invariant automatically is the central challenge. Approaches range from manual construction guided by domain knowledge, to automated inference using abstract interpretation, to counterexample-guided abstraction refinement (CEGAR) adapted to the parameterized setting. In the CEGAR approach, one begins with a coarse invariant, checks the finite abstract system, and if a spurious counterexample is found, refines the invariant to rule it out. Lesens, Halbwachs, and Raymond [LHR97] compute invariants for linear networks by iterating the composition and using an approximation that enforces convergence. Kesten, Pnueli, Shahar, and Zuck [KPSZ02] developed a deductive framework for network invariants, based on abstraction relations between systems, and applied it to a range of examples.

6.2 Counter Abstraction

The work of [PXZ02] combines abstraction with $(0, 1, \infty)$-*counter abstraction*, which tracks for each local state whether zero, one, or many processes occupy it. Since the number of local states is finite, the abstract system is finite. Its transitions are over-approximations of the concrete ones: for instance, when a process leaves a local state whose counter is ∞, the counter may become either 1 or remain ∞.

In the running example, the abstract state is a triple of counter values for i, w, and c, and the bad states are those in which the counter for c is ∞. A process can enter c only if the counter for c is 0, which sets it to 1, and leaving c sets it back to 0. The counter for c therefore never reaches ∞, and mutual exclusion holds in the abstract system, which has at most 27 states.

This finite abstraction is often coarse but captures enough structure to verify mutual exclusion and other coordination properties automatically, and the abstraction can be systematically refined when needed. The main contribution of [PXZ02] concerns liveness. The abstraction loses the identity of processes, so a fairness requirement for each individual process cannot be expressed directly. The method adds to the abstract system fairness conditions derived from the concrete system, which are strong enough to prove properties such as the absence of starvation.

6.3 Environment Abstraction

Environment abstraction [CTV06, CTV08] describes the system from the point of view of a single reference process. An abstract state consists of the local state of the reference process together with the truth values of a fixed set of predicates that describe the other processes in relation to it, for example whether some process with a smaller index is in its critical section, or

whether some other process holds a message addressed to the reference process. Because all processes are identical, it suffices to consider one reference process, and the abstract transition relation can be computed once from the template. The approach has been applied to mutual exclusion algorithms, such as the Bakery and Szymanski protocols, and to cache coherence protocols.

6.4 Compositional Reasoning for Protocols

Cache coherence protocols have been a main application of abstraction methods. McMillan [McM01] verified the FLASH protocol for an arbitrary number of processors by compositional model checking, combining symmetry, case splitting on the involved processors, and abstraction of the remaining ones, with lemmas supplied by the user. Chou, Mannava, and Park [CMP04] turned this approach into a simple and widely used method, often called the *CMP method*. A few processes are kept concrete, and all others are replaced by a single abstract process that can perform any action. This abstraction is usually too coarse, and spurious counterexamples are eliminated by adding noninterference lemmas, which constrain the actions of the abstract process and are themselves verified in the abstract model. Talupur and Tuttle [TT08] showed that many of the required lemmas can be derived from the *message flows* of a protocol, the sequences of messages exchanged to complete a transaction, which designers typically document.

Predicate abstraction has also been lifted to the parameterized setting. *Indexed predicates* contain free variables that range over process indices, and the abstraction computes invariants that are universally quantified over these variables [LB07]. This makes it possible to infer properties such as "for every process i, if i is in its critical section then the shared lock is held by i".

6.5 Thread-Modular Reasoning

For concurrent programs with an unbounded number of identical threads, *thread-modular* reasoning verifies a single thread against an assumption on the effect of the other threads, and checks that each thread respects this assumption [FQ03]. Since all threads are identical, a single assumption serves for all of them, which makes the method naturally parameterized. The precision of thread-modular reasoning can be increased by reasoning about k threads at a time instead of one, which yields a hierarchy of increasingly precise proof methods [HMP17]. *Proof spaces* [FKP15] provide a related framework, in which correctness proofs for an unbounded number of threads are built from a finite set of Hoare triples that are combined using rules that exploit the symmetry of the threads.

6.6 Relation to View Abstraction

The network invariant approach is closely related to view abstraction (Section 4) in spirit: both methods replace the unbounded environment of a process by a finite abstract summary. The key difference is in how that summary is constructed and updated. A network invariant is a single process that must be

supplied, or computed and refined, as a separate object, and its validity is checked by induction; view abstraction constructs the summary on the fly from the concrete reachability analysis, refining it incrementally as the precision parameter k increases. Counter abstraction and environment abstraction lie in between: the abstract domain is fixed in advance, but the abstract system is computed automatically from the template.

7 Deductive and Proof-Based Methods

Deductive approaches shift the verification task from automated state-space exploration to the construction of formal proofs, often assisted by theorem provers or proof assistants. The idea is to find invariants (properties that hold at every reachable state of the system for every n) and to prove that they are preserved by every system transition, regardless of the number of processes [OG76, Lam94]. Since the proof is carried out symbolically over an arbitrary n, this style of reasoning handles unbounded parameters without requiring enumeration of instances.

7.1 Inductive Invariants

We say that a set of configurations is an *inductive invariant* if it contains all initial configurations and is closed under the transition relation, that is, every successor of a configuration in the set is again in the set. An inductive invariant contains all reachable configurations, so if it is disjoint from the bad configurations, the system is safe. In the parameterized setting, invariants are expressed as formulas that quantify over process indices, and the checks are carried out for an arbitrary number n of processes.

For the running example, mutual exclusion is expressed by the formula stating that for all distinct indices i and j, processes i and j are not both in c. This formula is itself an inductive invariant. It holds initially, since all processes are in i. A step from i to w or from c to i does not create a process in c. A step in which some process k moves from w to c is only enabled when no other process is in c, so after the step, k is the only process in c. The argument does not depend on n or on k, and it is therefore a proof for all instances at once.

In realistic protocols, the property to be proved is rarely inductive by itself. If the universal guard of the running example is replaced by a loop in which a process checks the other processes one at a time, then a process may have checked some of the others while they were outside the critical section, and these processes may have entered it since. The property must then be *strengthened* with auxiliary assertions, for instance relating the progress of a process through its loop to the states of the processes it has already checked. Finding such strengthenings is the creative part of a deductive proof, and it is the step that the techniques in the rest of this section, and in Section 8, try to automate.

7.2 Assertional Reasoning and TLA+

Classic work by Owicki and Gries [OG76] laid the groundwork for assertional reasoning about concurrent programs: each pro-

cess is annotated with pre- and post-conditions, and an interference freedom check ensures that no process's execution invalidates another's assertions. The method was designed for a fixed, known number of processes, but the underlying proof discipline (inductive invariants together with noninterference) scales naturally to the parameterized case by carrying out the annotation and check for a symbolic, arbitrary process in a family.

Lamport's Temporal Logic of Actions [Lam94] and the specification language TLA+ [Lam02] provide a formal language for specifying and proving properties of concurrent and distributed systems in this spirit. A system is described by an initial predicate and a next-state relation, and parameters such as the set of processes are left as constants of the specification. The TLA+ proof system [CDL+12] supports machine-checked proofs of invariants for all values of these constants, while the TLC model checker is used to check the same specification for small, fixed values. Checking small instances with TLC has proved effective in finding subtle errors in industrial-scale protocols, for instance at Amazon Web Services [NRZ+15], and machine-checked proofs extend such results to all values of the parameters.

7.3 Invisible Invariants

A significant challenge for deductive methods is the discovery of inductive invariants: properties strong enough to be preserved under all transitions and to imply the desired safety property, but not so strong that they exclude reachable configurations. The *invisible invariants* technique of Pnueli, Ruah, and Zuck [PRZ01, APR+01] addresses this by automatically generating candidate invariants from small finite instances of the system and then checking whether they hold inductively for all n. The idea is that the right invariant is often already visible in small instances and can be generalized to the full parameterized family.

The procedure works as follows. First, the set of reachable configurations of an instance of a small size is computed symbolically, for example with BDDs. Second, this set is projected onto one or two processes, which yields a formula $\psi(i)$ or $\psi(i, j)$ describing the possible combinations of local states of these processes. Third, the formula is generalized to the candidate invariant "for all i, $\psi(i)$" or "for all distinct i and j, $\psi(i, j)$". Finally, the candidate is checked to be inductive and to imply the property. For a restricted class of systems, the last check can itself be carried out on an instance of bounded size, by a *small-model theorem* that bounds the size of a counterexample to inductiveness. In that case the whole procedure is automatic, and the invariant never needs to be seen by the user, which explains the name [ZP04].

In the running example, the reachable configurations of the instance of size two, projected onto both processes, give the formula $\psi(i, j)$ stating that i and j are not both in c. Its generalization is the invariant from the beginning of this section. This makes deductive verification partly automatic, at the cost of completeness: if the right invariant is not among those generated, the procedure does not terminate with a proof.

7.4 Interactive Proofs

More recent work integrates deductive reasoning with interactive tools, allowing arbitrarily complex invariants to be stated and verified with machine-checked proofs. The Ivy system [PMP+16], which we return to in Section 8, is representative of this direction. A *counterexample to induction* for a candidate invariant is a configuration that satisfies the candidate but has a successor that does not. When a candidate invariant is not inductive, Ivy presents such a counterexample. The user generalizes this configuration into a pattern that should be excluded, and the tool adds the corresponding conjunct to the invariant. The checks are carried out automatically, so the interaction focuses on the insight that the user brings, namely which features of a counterexample explain why it is unreachable.

Proof-based methods tend to be the most expressive and general of all approaches to parameterized verification, capable of handling protocols with complex data structures, arithmetic, and dynamic topology. Their principal limitation is the burden they place on the user: finding and expressing the right invariant requires deep insight into the protocol's correctness argument, and scaling to large or complex protocols remains demanding even with modern tool support.

8 SMT-Based Methods

The rise of powerful Satisfiability Modulo Theories (SMT) solvers over the past two decades has opened new avenues for parameterized verification. SMT-based methods encode system states, transitions, and properties as logical formulas and use highly optimized solvers to reason about them automatically [BSST09]. Their main advantage over purely deductive approaches is that the invariant search is largely automated; their main advantage over finite-state model checking is that sets of states are represented by formulas over rich theories, which makes them applicable to systems with rich data and large process counts.

8.1 Array-Based Systems

In *array-based systems* [GR10a, GR10b], the parameterized state is modeled as a collection of arrays indexed by process identifiers, and quantified formulas express properties over all processes simultaneously. For the running example, a single array s maps each process index to its local state. The initial condition is the formula stating that for every index i, $s[i] = \mathsf{i}$. The transition from w to c is described by a formula stating that there is an index k with $s[k] = \mathsf{w}$, that every index $j \neq k$ satisfies $s[j] \neq \mathsf{c}$, and that the new array is obtained from s by setting $s[k]$ to c. The bad configurations are described by the formula stating that there are two distinct indices i and j with $s[i] = \mathsf{c}$ and $s[j] = \mathsf{c}$.

The MCMT tool [GR10b] implements model checking for array-based systems via backward reachability in a quantified fragment of first-order logic, using SMT solvers to discharge the fixpoint conditions. The sets of configurations encountered

during backward reachability are represented by existentially quantified formulas, such as the formula for the bad configurations above. The predecessors of such a formula are again existentially quantified, provided that universal guards are handled appropriately; a common treatment instantiates the universal quantifier only with the indices mentioned in the formula, which over-approximates the system in a way that closely corresponds to monotonic abstraction (Section 5). The test that a new formula adds no configurations to those already found reduces to the satisfiability of a formula with an $\exists^*\forall^*$ quantifier prefix, which SMT solvers handle by instantiation.

In the running example, backward reachability starts from the formula for the bad configurations. A step from w to c by some process k can only lead to a bad configuration if the other process in c was already there before the step, which contradicts the guard. The other transitions lead from a configuration to a bad one only if the configuration is itself bad. The fixpoint is therefore reached immediately, and since no initial configuration satisfies the formula, the system is safe.

This approach handles systems in which each process has a local state drawn from an arbitrary (possibly infinite) domain, modeled as an interpreted theory in the SMT solver. Termination of backward reachability is not guaranteed in general, but it is guaranteed for classes of systems in which the models of the existential formulas, viewed as finite structures, are well-quasi-ordered, which connects the approach to the theory of Section 2.4 [GR10a]. The Cubicle model checker [CGK+12] follows the same principles, with a parallel implementation and a technique that uses finite instances of the system to guess approximations of the reachable states, with backtracking when an approximation turns out to be too coarse [CGK+13]. Array-based methods have seen success on cache coherence and distributed protocols, and have connections to the decidable array property fragments studied in program verification [BMS06].

8.2 IC3/PDR and Quantified Invariants

A prominent line of work in this space is *IC3/PDR* (Property Directed Reachability), originally developed for finite-state hardware verification [Bra11, EMB11] and later extended to the parameterized setting through quantified invariant inference. IC3/PDR works by incrementally constructing an inductive strengthening of the safety property. It maintains a sequence of formulas that over-approximate the sets of states reachable within increasing numbers of steps. It strengthens these formulas by blocking states from which a bad state can be reached, propagates facts from each formula to the next, and stops when two consecutive formulas coincide, at which point they form an inductive invariant. In the parameterized setting, these formulas are universally quantified over process identifiers, and SMT reasoning over the resulting quantified theories replaces the finite propositional checks of the original algorithm.

Karbyshev et al. [KBI+17] developed a variant of PDR that infers universally quantified invariants. The *diagram* of a state is

an existential formula that describes the state up to the identity of its elements. When a state must be blocked, the algorithm blocks its diagram. Blocking a diagram excludes the state together with all larger states that contain it, which makes the method similar to backward reachability over upward-closed sets. When it terminates, the algorithm returns a counterexample, a universal inductive invariant, or a proof that no such invariant exists. Gurfinkel, Shoham, and Meshman [GSM16] exploit the symmetry of parameterized systems to reduce the inference of quantified invariants to the inference of invariants for a small number of representative processes, a task that can be delegated to an SMT-based model checker.

8.3 Decidable Logics and Ivy

Tools such as Ivy [PMP+16] build on these ideas to provide interactive and automated frameworks for specifying and verifying distributed protocols. Ivy allows users to specify systems in a first-order language and restricts the verification conditions to the *effectively propositional* fragment EPR, in which formulas have a quantifier prefix of the form $\exists^*\forall^*$ and contain no function symbols, extended with function symbols whose use satisfies an acyclicity condition. Satisfiability in this fragment is decidable, and every satisfiable formula has a finite model whose size is bounded by the formula. Consequently, every failed check yields a small, finite counterexample, which the user can inspect, as described in Section 7. The bound on the size of models plays a role similar to a cutoff: it limits the number of processes, messages, and values that need to be considered.

Many protocols do not fit into EPR directly. Padon et al. [PLSS17] showed that complex consensus protocols, including several variants of Paxos, can be verified within EPR by introducing derived relations that keep the verification conditions in the decidable fragment. In practice, this workflow has successfully verified a range of distributed protocols, including variants of Paxos, two-phase commit, and leader election. The inference of invariants for such protocols has also been automated. One approach generalizes invariants of small finite instances to quantified invariants and checks them for all sizes [MGJ+19], in the spirit of invisible invariants; another searches for quantified formulas that separate reachable states from states that lead to an error [KPIA20].

8.4 Threshold-Based Distributed Algorithms

Fault-tolerant distributed algorithms form a further class of parameterized systems, in which the parameters include not only the number n of processes but also an upper bound t on the number of faulty processes. Processes typically wait until they have received messages from more than a threshold of other processes, such as $(n + t)/2$, before taking a step. Such algorithms were first verified by abstraction-based parameterized model checking [JKS+13], and have since been modeled as *threshold automata*, in which transitions are guarded by linear constraints over message counters and parameters [KVW17]. A key result is a *short*

counterexample property: if a threshold automaton violates a safety or liveness property for some values of the parameters, then it does so in a run whose length, measured in a suitable sense, is bounded independently of the parameters. Bounded model checking with an SMT solver over the parameters then suffices for all parameter values [KVW17, KLVW17].

8.5 Discussion

The relationship between SMT-based methods and the other approaches in this chapter is instructive. SMT methods can be seen as automating the invariant discovery step that deductive methods (Section 7) leave to the user, while using a logical representation of state that complements the regular-language and upward-closed-set representations of Section 5: logical formulas easily express data and arithmetic, while automata easily express the ordering of processes in a configuration. The tradeoff is decidability: the quantified theories used by SMT-based tools are generally undecidable, and the tools rely on heuristics, user interaction, or restriction to decidable fragments to terminate. SMT-based methods are thus more automatic than purely deductive approaches, but they lack the termination guarantees of cutoff and monotonic abstraction methods.

9 Applications to Weak Memory Models

Modern multiprocessor architectures do not guarantee Sequential Consistency (SC) [Lam79]: for performance reasons, processors reorder memory operations in ways that are invisible to a single thread but observable between concurrently executing threads. The resulting *weak* (or *relaxed*) *memory models* (WMMs) include Total Store Order (TSO), which underlies x86 and SPARC processors [OSS09, SSO+10], and Partial Store Order (PSO), among others [AMT14]. Verifying concurrent programs under such models is substantially harder than under SC, because each thread is equipped with a buffer that decouples writes from their visibility to other threads, making the state space infinite even for finite-state programs. Parameterized verification under WMMs is harder still: one must reason simultaneously about an unbounded number of threads, each carrying an unbounded buffer.

Parameterized questions arise naturally in this setting. Synchronization primitives, such as locks, barriers, and mutual exclusion protocols, are written to be used by any number of threads, and the same code is executed by all of them. A correct implementation must work for every number of threads, and on every architecture on which it is deployed, which means under the memory model of that architecture.

The connection to the techniques covered in earlier sections is direct. Backward reachability (Section 2.4) applies when the system's transition relation is monotone with respect to a suitable well-quasi-ordering on configurations, and monotonic abstraction (Section 5) enforces monotonicity by over-approximation when it does not hold. The key contribution of the weak-memory line of work is to identify the right semantics and the right ordering

Thread 1	Thread 2
$x := 1$	$y := 1$
$r_1 := y$	$r_2 := x$

Figure 3: The store buffering program. The variables x and y are initially 0. Under SC, at least one of the two reads returns 1. Under TSO, both reads may return 0.

to make this apply, even though store and load buffers introduce data that naively breaks monotonicity.

9.1 The TSO Model and the Dual Semantics

Under the classical TSO semantics, each thread is equipped with an unbounded perfect FIFO *store buffer*. When a thread writes to a variable, the write is appended to its store buffer rather than committed to memory immediately; values propagate to shared memory non-deterministically in FIFO order. A thread reading a variable first checks its own buffer for the most recent pending write, and falls back to main memory if none is found. Fence instructions force the buffer to drain before execution continues.

The program of Figure 3 illustrates the difference from SC. Under SC, one of the two writes is executed first, so at least one of the reads returns the value 1. Under TSO, thread 1 can place its write to x in its store buffer, and thread 2 can do the same with its write to y. Each thread then reads the other variable from memory, where it still has the value 0, so the read of y by thread 1 returns 0, and the read of x by thread 2 returns 0. Only afterwards are the two writes propagated to memory. This behavior breaks mutual exclusion protocols such as Dekker's and Peterson's algorithms, which rely on the fact that a thread that has announced its intention by a write will be seen by the others. Fences are needed to restore correctness, and verification under TSO is used, among other things, to check that enough fences have been inserted. We say that a program is *robust* if all its behaviors under TSO are also possible under SC. Checking robustness is a related verification problem [BDM13].

Decidability of safety verification under TSO for a fixed number of threads was established by encoding store buffers as lossy channels and applying the theory of well-structured transition systems (WSTS) [ABBM10]. However, these encodings are complex: each channel message must carry a full memory snapshot, and the reduction involves non-deterministic guessing that leads to significant state-space explosion in practice.

An alternative approach introduces a *dual* semantics for TSO [AABN16, AABN18]. Rather than store buffers, the dual model equips each thread with a *load buffer*: values propagate non-deterministically from shared memory into load buffers, and reads are served from the head of the load buffer rather than from memory directly. Writes, in contrast, are committed to memory immediately and tagged in the buffer as *own* messages. In the program of Figure 3, the initial values of x and y can propagate

into the load buffers of threads 2 and 1, respectively, before any write takes place. Both writes are then committed to memory, and each thread reads the old value 0 from its load buffer. Thus, instead of delaying the writes, the dual semantics delays the reads. The dual semantics is provably equivalent to the classical one, in the sense that the two models reach the same sets of local states. It offers significant practical advantages. Because load buffers behave as lossy channels in a straightforward sense, the WSTS framework applies with far less overhead than in the store-buffer encoding, yielding a simpler decidability proof and a more efficient verification algorithm.

9.2 Parameterized Verification under TSO

The dual-semantics approach extends naturally to parameterized verification. In the parameterized setting, the number of threads is a free parameter, and one asks whether a bad local state is reachable for *any* number of threads. Parameterized configurations are ordered by a well-quasi-ordering that combines the subword ordering on load buffer contents with a matching on threads, in which each thread of the smaller configuration is mapped to a distinct thread of the larger one; the transition system is shown to be monotone with respect to this ordering; and backward reachability over upward-closed sets of configurations terminates by Higman's lemma. This yields the first decidability result for parameterized verification of concurrent programs under a weak memory model [AABN16,AABN18]. Moreover, experiments with a prototype tool show that parameterized verification is often *more* efficient than bounding the number of threads to a concrete value: the sets of minimal elements produced by the backward analysis are typically much smaller than the configuration sets generated by bounded-instance analyses. Already for a fixed number of threads, the load-buffer semantics gives large gains in efficiency and scalability compared with verification tools based on the store-buffer semantics, on standard mutual exclusion and memory-model benchmarks [AABN18].

The observation that load buffers, unlike store buffers, are compatible with monotonicity is the central point. Adding a message to a store buffer can change the value that its thread reads, since the thread reads its own most recent write, and can therefore disable a transition. Adding a message to a load buffer only adds the possibility of reading an old value, which the thread is free to skip. Larger configurations can thus simulate smaller ones, as required by the theory of Section 2.4.

Complexity of parameterized TSO verification. While the dual-semantics approach establishes decidability, it leaves open the precise computational complexity of parameterized TSO verification. This question was settled in [AAR20], where it is shown that the reachability problem for a parameterized system of identical processes under TSO is PSpace-complete.

The PSpace upper bound is obtained by combining the load-buffer (dual) semantics with a careful observation about what information needs to be tracked: it suffices to maintain, for each

process, the set of write operations it has issued together with the current value it observes for each variable, rather than the full buffer contents. This yields a polynomial-space abstraction that is both sound and complete for reachability. The matching lower bound is established by a reduction from the reachability problem for 1-safe Petri nets, and the reduction relies on the ability of processes to read the *initial values* of variables from memory. This dependence on initial values is not merely an artifact of the proof: when processes are forbidden from reading initial memory values, the reachability problem drops to polynomial time, since the relevant information can be captured by a polynomial-sized abstraction tracking only the set of writes issued across all processes. A further variant is studied in which processes may perform *atomic read-modify-write* (RMW) operations; it is shown that adding RMW makes the problem non-primitive recursive, matching the known hardness of reachability for general lossy channel systems. Together, these results give a precise and tight complexity picture for parameterized TSO verification across different fragments of the model. Parameterized verification has also been studied for other memory models; for instance, it is PSPACE-complete under the release-acquire semantics of C11 [KGMC22].

9.3 The Intel x86 (eTSO) Model

Real Intel x86 processors implement a memory model that goes beyond TSO. The formal model eTSO [RMV22] accounts for multiple memory types, namely write-back (wb), write-combining (wc), and strong-uncacheable, as well as non-temporal writes (ntw). Write-back writes behave as in TSO, while non-temporal writes on different variables may overtake one another, similarly to PSO. This makes eTSO strictly more relaxed than TSO, and the interaction between wb and ntw operations introduces subtle orderings that do not arise in either TSO or PSO alone. Reachability under eTSO for a fixed number of threads is in general undecidable, but decidability can be recovered by bounding the number of *alternations* between wb and ntw write stages per thread [AAB$^+$24]. A run is k-alternation bounded if the writes of each thread can be partitioned into at most k blocks, alternating between blocks containing only wb writes and blocks containing only ntw writes. Even under this restriction, executions may involve unbounded sequences of operations within each block, and complex shuffles of writes across blocks are permitted.

Parameterized verification of this k-alternation bounded problem was established in [AAB$^+$26]. The proof introduces a new transition system, called the *NTWLoad* system, which combines the load-buffer technique from the dual TSO semantics (to handle wb writes while preserving monotonicity) with per-variable store buffers for ntw writes. Each eTSO thread is simulated by $2k$ concurrent *stage threads*, alternating between wb and ntw stages, with a finite interface that enforces the required causal dependencies across stages.

A central technical difficulty is, as explained above, that TSO store buffers are not monotone with respect to the subword ordering. This difficulty is circumvented by switching wb writes to

the load-buffer representation, in which monotonicity is restored, while ntw writes retain their store-buffer treatment. The causal dependencies across stages are maintained through an explicit bookkeeping mechanism. For each stage, it tracks speculated write values, expected termination labels, and obligations that must be discharged during execution.

This bookkeeping mechanism introduces a further complication: the resulting ordering on configurations is not merely a well-quasi-ordering (WQO) but must be a *better quasi-ordering* (BQO) [Mar94]. BQOs are strictly stronger than WQOs and enjoy additional closure properties, in particular under the powerset construction, that WQOs do not. The ordering on NTWLoad configurations is built by combining a subword ordering on buffer contents with an equality constraint on stage information and a powerset-style matching on thread sets; showing this composite ordering is a BQO requires appealing to the closure of BQOs under products, word embeddings, and powerset operations. The transition relation is then shown to be monotone with respect to this BQO, placing the system within the WSTS framework and yielding decidability of backward reachability. BQOs had earlier been used in verification to obtain efficient symbolic representations for timed Petri nets [AN00, Abd10]. To the best of our knowledge, the result of [AAB+26] is the first application of BQO-based well-structured systems to prove decidability of a program verification problem.

10 Parameterized Verification of Quantum Circuits

The preceding sections have developed parameterized verification in the classical setting: processes communicate through shared memory or message passing, the parameter n counts the number of processes, and correctness is expressed as a safety or liveness property over an infinite family of transition systems. A structurally analogous challenge arises in quantum computing. Most useful quantum algorithms, such as Shor's factoring algorithm [Sho97], Grover's search [Gro96], and the Quantum Fourier Transform, are *parameterized by input size*: a quantum *program* takes n as input and dynamically generates the corresponding n-qubit circuit. Verifying such a program requires establishing correctness for all values of n simultaneously, a goal that parallels classical parameterized verification but requires substantially different technical machinery.

10.1 Quantum States and Circuits

We recall only the notions needed in this section; a standard introduction is [NC10]. The state of n qubits is a vector of 2^n complex *amplitudes*, one for each *computational basis state*, that is, for each bit string b of length n. We write $|b\rangle$ for the basis state that corresponds to b, and write a state as a sum $\sum_b a_b|b\rangle$, where the amplitudes a_b satisfy the normalization condition that the squares of their absolute values sum to one. A *gate* applies a unitary transformation to one or a few qubits. The Hadamard gate H maps $|0\rangle$ to $(|0\rangle + |1\rangle)/\sqrt{2}$ and $|1\rangle$ to $(|0\rangle - |1\rangle)/\sqrt{2}$. The controlled-not gate CNOT acts on a control qubit and a target



Figure 4: Left: the circuit that prepares the GHZ state on three qubits, consisting of a Hadamard gate followed by two CNOT gates (control marked by a dot, target by a circle). Right: the GHZ state on two qubits represented as a binary tree; each path from the root to a leaf selects the values of the qubits, and the leaf stores the corresponding amplitude.

qubit, and flips the target if the control is 1. A *circuit* is a sequence of gates, and its effect on a state is the composition of the effects of its gates.

As a running example for this section, consider the parameterized program that prepares the *GHZ state* on n qubits. Starting from the state $|0^n\rangle$, in which all qubits are 0, it applies H to the first qubit and then, for each i from 1 to $n-1$, a CNOT with control i and target $i+1$. The result is the state $(|0^n\rangle + |1^n\rangle)/\sqrt{2}$, in which all qubits are 0 or all are 1, each with probability one half. Figure 4 shows the circuit for $n = 3$.

10.2 The Verification Problem

The correctness criterion for quantum programs is characteristically different from the safety and liveness properties of Sections 3–9. Rather than asking whether a bad state is ever reached, one asks whether the program computes the right function: given a set of quantum state pairs (p, q), does the program, for every input size n, correctly map input state p to output state q? This *relational* (or *functional*) specification captures input-output correctness across all parameterized instances simultaneously. For the GHZ program, the specification consists of the pairs $(|0^n\rangle, (|0^n\rangle + |1^n\rangle)/\sqrt{2})$, one for each n. The specification also subsumes *equivalence checking* between two parameterized programs, a property of direct practical importance for verifying that an optimized circuit implementation is functionally identical to a reference specification, regardless of n.

Existing approaches to quantum program verification have made notable progress but fall short of fully automatic treatment of general parameterized circuits. Deductive approaches, based on quantum Hoare logics [Yin11] or on proof assistants in which circuit optimizations are verified once and for all [HRH+21], can handle parameterized programs but require user guidance, while symbolic techniques such as path sums [Amy19] automatically verify circuits of a fixed size. The Qbricks project [CBB+21] develops a deductive framework for circuit-building quantum programs with rich specifications; it achieves a high degree of automation, but still requires user-provided annotations such as invariants. Automata-based approaches [CCL+23b, CCL+23a, ACC+25] represent sets of quantum states by tree automata and compute the effect of each gate automatically, which makes verification fully automatic for

circuits of a fixed size, and they have been extended to programs with control flow [CCH+25]. A fully automatic framework for general parameterized quantum programs was an open problem until recently [ACH+26].

10.3 Representing Infinite Families of Quantum States

The central challenge in all parameterized verification is finding a finite symbolic representation of an infinite family of system states, one state space for each value of n. In the classical setting, regular languages over a finite alphabet serve this role (Section 5): a single finite automaton compactly represents all reachable configurations across all instance sizes. In the quantum setting, a parameterized program generates an infinite family of state vectors, one for each n, whose size grows exponentially with n but which have a regular recursive structure.

The framework of [ACH+26] achieves this through a new automata model called *synchronized weighted tree automata* (SWTAs). The starting point is the observation that an n-qubit quantum state can be naturally represented as a binary tree of depth n, as in Figure 4: each internal node corresponds to a qubit, and each leaf stores the complex amplitude associated with a particular computational basis state. Standard tree automata recognize sets of such trees, providing a symbolic representation of sets of quantum states that is analogous to finite automata over words. SWTAs extend this with two additional features. First, *colors* attached to transitions serve as synchronization labels: all transitions applied at the same level of a tree must carry the same color, which enforces the level-wise uniformity that families of quantum states require. A tree is thereby associated with a sequence of colors, one for each level, and related input and output trees share the same color sequence; this is the mechanism by which input-output pairs are linked. Second, *weights* on transitions encode complex amplitudes, enabling the automaton to represent not merely which quantum states are possible, but what amplitude each state carries. Together, these extensions yield a model that defines a *function* from color sequences to quantum states, rather than simply a set of states. This functional view is essential for relational verification: it allows a specification to assert what output state is expected for each input state, across all input sizes simultaneously.

The GHZ example illustrates the gain in succinctness. The tree for the GHZ state on n qubits has 2^n leaves, but its structure is simple: the left subtree of the root contains a single nonzero amplitude at its leftmost leaf, the right subtree contains a single nonzero amplitude at its rightmost leaf, and all other subtrees are zero. A handful of automaton states, one for each kind of subtree, describes this structure at every depth, and a weight of $1/\sqrt{2}$ at the root gives the amplitudes. The infinite family of GHZ states, one for each n, is therefore represented by a single small automaton, in the same way as the infinite family of token-passing configurations $\mathsf{n}^*\mathsf{t}\mathsf{n}^*$ is represented by a small finite automaton in Section 5.

10.4 Transducers for Quantum Gate Semantics

In regular model checking, the transition relation of a parameterized system is encoded as a finite-state transducer, and reachability reduces to computing the transitive closure of that transducer (Section 5). The quantum analogue replaces the transition relation with a *quantum gate*, and the transducer must encode the linear transformation that the gate applies to the quantum state tree. The framework of [ACH+26] introduces a class of transducers that operate directly on SWTAs: given an input SWTA representing a family of quantum states, applying a gate transducer produces an output SWTA representing the family of states after the gate has been applied.

A central concern in any symbolic framework for quantum computation is *succinctness*: a quantum gate on n qubits is a $2^n \times 2^n$ unitary matrix, and a naive representation is exponential in n. The transducer representation avoids this blow-up. As an illustration, the transducer for the Quantum Fourier Transform on n qubits has a number of states that is *polynomial* in n, exploiting the recursive structure of the QFT to obtain an exponentially more compact representation than a matrix encoding would require. This mirrors the advantage of transducers over explicit transition tables in classical regular model checking.

A further key ingredient is a *composition operator* for transducers. Given transducers for two sub-circuits, the operator automatically constructs a transducer for their sequential composition. Starting from transducers for individual gates, repeated composition assembles the transducer for an entire circuit without user intervention. The operator extends to *parameterized programs*: given a transducer for a finite circuit constituting one layer of a parameterized algorithm, composition generates a transducer for the full parameterized version with an arbitrary number of such layers. This step lifts finite-circuit verification to parameterized verification, and it is the quantum analogue of computing the transitive closure of a transducer in classical regular model checking.

10.5 Verification Algorithms and Experimental Results

With SWTAs as the specification formalism and transducers as the program model, verification compares the functions from color sequences to quantum states that SWTAs define. We write $[\![\mathcal{A}]\!]$ for the function defined by an SWTA $\mathcal{A}$. Let $\mathcal{A}$ be the SWTA that specifies the expected outputs, and let $\mathcal{A}'$ be the SWTA obtained by applying the program transducer to the input specification. Verification then reduces to *functional inclusion*, that is, to checking that $[\![\mathcal{A}']\!]$ is contained in $[\![\mathcal{A}]\!]$ when both are viewed as sets of pairs of a color sequence and a state. Equivalence checking between two parameterized programs reduces to *functional equivalence* of the output SWTAs they produce when applied to a complete set of linearly independent inputs; correctness of this reduction follows from the fact that quantum programs implement linear transformations, so agreement on a complete input basis implies agreement on all inputs. Functional inclusion and functional equivalence of SWTAs are decidable, and the frame-

work provides algorithms for both. In contrast, inclusion and equivalence of the *languages* of SWTAs, viewed as sets of trees, are undecidable, which is one reason why the functional view is essential [ACH+26].

The implementation, called AutoQ-Para [ACH+26], was evaluated on a diverse set of benchmarks spanning algorithmic, arithmetic, error-correction, and simulation circuits. It successfully verified the Bernstein-Vazirani algorithm, a ripple-carry adder, a syndrome extraction circuit from a quantum error-correcting code, and optimized implementations of Grover's search and Hamiltonian simulation, comparing them against their unoptimized reference counterparts. Verification times ranged from 14 milliseconds to 11 seconds, and to the best of our knowledge these are the first fully automatic parameterized verifications of circuits of this kind.

10.6 Connections to Classical Parameterized Verification

The structural parallels with classical parameterized verification are instructive. In both settings the parameter n indexes an infinite family of systems, and the goal is to verify a property of the entire family from a finite symbolic representation. Regular languages and SWTAs both fill this role, using words for classical configurations and trees for quantum states. Transducers encode transition relations in the classical case and gate semantics in the quantum case, and computing transitive closures or repeated compositions is the operation that lifts finite reasoning to the parameterized family in both. The functional inclusion check mirrors the final step of the classical analyses, amounting to verifying that no reachable output state lies outside the specification. What distinguishes the quantum setting is the algebraic structure of the state space: quantum states carry complex amplitudes subject to normalization and unitarity constraints, and gates are unitary linear transformations rather than arbitrary relations. The SWTA model is designed to respect this structure, with weights on transitions as the mechanism for tracking amplitudes within an automata-theoretic framework. The result can be seen as an extension of regular model checking to quantum programs, adapted to the linear-algebraic semantics of quantum computation.

11 Industrial Applications

12 Relations to Other Chapters

13 Conclusion and Open Problems

Parameterized verification asks for guarantees that hold for every number of components, a question that is undecidable in general. The techniques surveyed in this chapter respond to this difficulty in different ways, but the same few ideas recur across all of them: a finite symbolic representation of infinite families of states, monotonicity with respect to a well-quasi-ordering, and the observation that the behavior of a large instance is often already visible in a small one. The applications to weak memory

models and to quantum circuits show that these ideas carry over to settings that look very different from the protocols for which they were first developed.

Several problems remain open or only partially solved. Liveness properties under fairness are much less well supported than safety properties, both in theory and in tools, since abstractions that preserve safety often lose the information needed for fairness. For weak memory models, parameterized verification is well understood for TSO and some of its extensions, but the more relaxed models of ARM and POWER processors, and the language-level models of C11 and Java, are only partly covered. For distributed protocols, the automatic inference of invariants with alternating quantifiers, which are needed for many consensus protocols, remains difficult, and current tools handle it by restricting the logic or by relying on the user. In the quantum setting, the automata-based approach covers parameterized circuits, and its extension to programs with measurement, classical control, and noise is a natural next step. Finally, the parameterized view applies to further classes of systems built from many copies of the same component, such as multi-agent systems and large-scale distributed services, where the same questions about correctness for any number of components arise.

References

[AAB+24] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, K. Narayan Kumar, and Prakash Saivasan. Verification under Intel-x86 with persistency. *Proceedings of the ACM on Programming Languages*, 8(PLDI):1189–1212, 2024.

[AAB+26] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, K. Narayan Kumar, and Prakash Saivasan. Parametrised verification of Intel-x86 programs. *Proceedings of the ACM on Programming Languages*, 10(POPL):1094–1122, 2026.

[AABN16] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, and Tuan Phong Ngo. The benefits of duality in verifying concurrent programs under TSO. In *Proceedings of CONCUR 2016*, volume 59 of *Leibniz International Proceedings in Informatics*, pages 5:1–5:15. Schloss Dagstuhl, 2016.

[AABN18] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Ahmed Bouajjani, and Tuan Phong Ngo. A load-buffer semantics for total store ordering. *Logical Methods in Computer Science*, 14(1), 2018.

[AAD+06] Dana Angluin, James Aspnes, Zoë Diamadi, Michael J. Fischer, and René Peralta. Computation in networks of passively mobile finite-state sensors. *Distributed Computing*, 18(4):235–253, 2006.

[AAR20] Parosh Aziz Abdulla, Mohamed Faouzi Atig, and Rojin Rezvan. Parameterized verification under TSO is PSPACE-complete. *Proceedings of the ACM on Programming Languages*, 4(POPL):1–29, 2020.

[ABBM10] Mohamed Faouzi Atig, Ahmed Bouajjani, Sebastian Burckhardt, and Madanlal Musuvathi. On the verification

problem for weak memory models. In *Proceedings of POPL 2010*, pages 7–18. ACM, 2010.

[Abd10] Parosh Aziz Abdulla. Well (and better) quasi-ordered transition systems. *The Bulletin of Symbolic Logic*, 16(4):457–515, 2010.

[Abd12] Parosh Aziz Abdulla. Regular model checking. *International Journal on Software Tools for Technology Transfer*, 14(2):109–118, 2012.

[Abd21] Parosh Aziz Abdulla. Regular model checking: Evolution and perspectives. In Ernst-Rüdiger Olderog, Bernhard Steffen, and Wang Yi, editors, *Model Checking, Synthesis, and Learning*, volume 13030 of *Lecture Notes in Computer Science*, pages 78–96. Springer, 2021.

[ABHDR08] Parosh Aziz Abdulla, Noomene Ben Henda, Giorgio Delzanno, and Ahmed Rezine. Handling parameterized systems with non-atomic global conditions. In *Proceedings of VMCAI 2008*, volume 4905 of *Lecture Notes in Computer Science*, pages 22–36. Springer, 2008.

[ACC+25] Parosh Aziz Abdulla, Yo-Ga Chen, Yu-Fang Chen, Lukáš Holík, Ondřej Lengál, Jyun-Ao Lin, Fang-Yi Lo, and Wei-Lun Tsai. Verifying quantum circuits with level-synchronized tree automata. *Proceedings of the ACM on Programming Languages*, 9(POPL):923–953, 2025.

[ACD+10] Parosh Aziz Abdulla, Yu-Fang Chen, Giorgio Delzanno, Frédéric Haziza, Chih-Duo Hong, and Ahmed Rezine. Constrained monotonic abstraction: A CEGAR for parameterized verification. In *Proceedings of CONCUR 2010*, volume 6269 of *Lecture Notes in Computer Science*, pages 86–101. Springer, 2010.

[ACH+26] Parosh Aziz Abdulla, Yu-Fang Chen, Michal Hečko, Lukáš Holík, Ondřej Lengál, Jyun-Ao Lin, and Ramanathan S. Thinniyam. Parameterized verification of quantum circuits. *Proceedings of the ACM on Programming Languages*, 10(POPL):2021–2050, 2026.

[AČJT96] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. General decidability theorems for infinite-state systems. In *Proceedings of LICS 1996*, pages 313–321. IEEE Computer Society, 1996.

[AČJT00] Parosh Aziz Abdulla, Kārlis Čerāns, Bengt Jonsson, and Yih-Kuen Tsay. Algorithmic analysis of programs with well quasi-ordered domains. *Information and Computation*, 160(1–2):109–127, 2000.

[ADBHR07] Parosh Aziz Abdulla, Giorgio Delzanno, Noomene Ben Henda, and Ahmed Rezine. Regular model checking without transducers (on efficient verification of parameterized systems). In *Proceedings of TACAS 2007*, volume 4424 of *Lecture Notes in Computer Science*, pages 721–736. Springer, 2007.

[ADR09] Parosh Aziz Abdulla, Giorgio Delzanno, and Ahmed Rezine. Approximated parameterized verification of infinite-state processes with global conditions. *Formal Methods in System Design*, 34(2):126–156, 2009.

[AHH13] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. All for the price of few: Parameterized verification

through view abstraction. In *Proceedings of VMCAI 2013*, volume 7737 of *Lecture Notes in Computer Science*, pages 476–495. Springer, 2013.

[AHH14] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. Block me if you can! Context-sensitive parameterized verification. In *Proceedings of SAS 2014*, volume 8723 of *Lecture Notes in Computer Science*, pages 1–17. Springer, 2014.

[AHH16] Parosh Aziz Abdulla, Frédéric Haziza, and Lukáš Holík. Parameterized verification through view abstraction. *International Journal on Software Tools for Technology Transfer*, 18(5):495–516, 2016.

[AJK16] Simon Außerlechner, Swen Jacobs, and Ayrat Khalimov. Tight cutoffs for guarded protocols with fairness. In *Proceedings of VMCAI 2016*, volume 9583 of *Lecture Notes in Computer Science*, pages 476–494. Springer, 2016.

[AJKR14] Benjamin Aminof, Swen Jacobs, Ayrat Khalimov, and Sasha Rubin. Parameterized model checking of token-passing systems. In *Proceedings of VMCAI 2014*, volume 8318 of *Lecture Notes in Computer Science*, pages 262–281. Springer, 2014.

[AJMd02] Parosh Aziz Abdulla, Bengt Jonsson, Pritha Mahata, and Julien d'Orso. Regular tree model checking. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 555–568. Springer, 2002.

[AJNd02] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Julien d'Orso. Regular model checking made simple and efficient. In *Proceedings of CONCUR 2002*, volume 2421 of *Lecture Notes in Computer Science*, pages 116–131. Springer, 2002.

[AJNd03] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Julien d'Orso. Algorithmic improvements in regular model checking. In *Proceedings of CAV 2003*, volume 2725 of *Lecture Notes in Computer Science*, pages 236–248. Springer, 2003.

[AJNS04] Parosh Aziz Abdulla, Bengt Jonsson, Marcus Nilsson, and Mayank Saksena. A survey of regular model checking. In *Proceedings of CONCUR 2004*, volume 3170 of *Lecture Notes in Computer Science*, pages 35–48. Springer, 2004.

[AK86] Krzysztof R. Apt and Dexter C. Kozen. Limits for automatic verification of finite-state concurrent systems. *Information Processing Letters*, 22(6):307–309, 1986.

[AKR+18] Benjamin Aminof, Tomer Kotek, Sasha Rubin, Francesco Spegni, and Helmut Veith. Parameterized model checking of rendezvous systems. *Distributed Computing*, 31(3):187–222, 2018.

[AMT14] Jade Alglave, Luc Maranget, and Michael Tautschnig. Herding cats: Modelling, simulation, testing, and data mining for weak memory. *ACM Transactions on Programming Languages and Systems*, 36(2):1–74, 2014.

[Amy19] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. *Electronic Proceedings in Theoretical Computer Science*, 287:1–21, 2019.

[AN00] Parosh Aziz Abdulla and Aletta Nylén. Better is better than well: On efficient verification of infinite-state systems. In *Proceedings of LICS 2000*, pages 132–140. IEEE Computer Society, 2000.

[APR+01] Tamarah Arons, Amir Pnueli, Sitvanit Ruah, Ying Xu, and Lenore Zuck. Parameterized verification with automatically computed inductive assertions. In *Proceedings of CAV 2001*, volume 2102 of *Lecture Notes in Computer Science*, pages 221–234. Springer, 2001.

[AST18] Parosh Aziz Abdulla, A. Prasad Sistla, and Muralidhar Talupur. Model checking parameterized systems. In Edmund M. Clarke, Thomas A. Henzinger, Helmut Veith, and Roderick Bloem, editors, *Handbook of Model Checking*, pages 685–725. Springer, 2018.

[BCG89] Michael C. Browne, Edmund M. Clarke, and Orna Grumberg. Reasoning about networks with many identical finite state processes. *Information and Computation*, 81(1):13–31, 1989.

[BDM13] Ahmed Bouajjani, Egor Derevenetc, and Roland Meyer. Checking and enforcing robustness against TSO. In *Proceedings of ESOP 2013*, volume 7792 of *Lecture Notes in Computer Science*, pages 533–553. Springer, 2013.

[BHV04] Ahmed Bouajjani, Peter Habermehl, and Tomáš Vojnar. Abstract regular model checking. In *Proceedings of CAV 2004*, volume 3114 of *Lecture Notes in Computer Science*, pages 372–386. Springer, 2004.

[BJK+15] Roderick Bloem, Swen Jacobs, Ayrat Khalimov, Igor Konnov, Sasha Rubin, Helmut Veith, and Josef Widder. *Decidability of Parameterized Verification*. Synthesis Lectures on Distributed Computing Theory. Morgan & Claypool, 2015.

[BJNT00] Ahmed Bouajjani, Bengt Jonsson, Marcus Nilsson, and Tayssir Touili. Regular model checking. In *Proceedings of CAV 2000*, volume 1855 of *Lecture Notes in Computer Science*, pages 403–418. Springer, 2000.

[BLW03] Bernard Boigelot, Axel Legay, and Pierre Wolper. Iterating transducers in the large. In *Proceedings of CAV 2003*, volume 2725 of *Lecture Notes in Computer Science*, pages 223–235. Springer, 2003.

[BLW05] Ahmed Bouajjani, Axel Legay, and Pierre Wolper. Handling liveness properties in (ω-)regular model checking. *Electronic Notes in Theoretical Computer Science*, 138(3):101–115, 2005.

[BMS06] Aaron R. Bradley, Zohar Manna, and Henny B. Sipma. What's decidable about arrays? In *Proceedings of VMCAI 2006*, volume 3855 of *Lecture Notes in Computer Science*, pages 427–442. Springer, 2006.

[Bra11] Aaron R. Bradley. SAT-based model checking without unrolling. In *Proceedings of VMCAI 2011*, volume 6538 of *Lecture Notes in Computer Science*, pages 70–87. Springer, 2011.

[BSST09] Clark Barrett, Roberto Sebastiani, Sanjit A. Seshia, and Cesare Tinelli. Satisfiability modulo theories. In *Handbook of Satisfiability*, volume 185 of *Frontiers in Artificial Intelligence and Applications*, pages 825–885. IOS Press, 2009.

[BT02] Ahmed Bouajjani and Tayssir Touili. Extrapolating tree transformations. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 539–554. Springer, 2002.

[CBB+21] Christophe Chareton, Sébastien Bardin, François Bobot, Valentin Perrelle, and Benoît Valiron. An automated deductive verification framework for circuit-building quantum programs. In *Proceedings of ESOP 2021*, volume 12648 of *Lecture Notes in Computer Science*, pages 148–177. Springer, 2021.

[CCH+25] Yu-Fang Chen, Kai-Min Chung, Min-Hsiu Hsieh, Wei-Jia Huang, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. AutoQ 2.0: From verification of quantum circuits to verification of quantum programs. In *Proceedings of TACAS 2025*, Lecture Notes in Computer Science, pages 87–108. Springer, 2025.

[CCL+23a] Yu-Fang Chen, Kai-Min Chung, Ondřej Lengál, Jyun-Ao Lin, and Wei-Lun Tsai. AutoQ: An automata-based quantum circuit verifier. In *Proceedings of CAV 2023*, volume 13966 of *Lecture Notes in Computer Science*, pages 139–153. Springer, 2023.

[CCL+23b] Yu-Fang Chen, Kai-Min Chung, Ondřej Lengál, Jyun-Ao Lin, Wei-Lun Tsai, and Di-De Yen. An automata-based framework for verification and bug hunting in quantum circuits. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1218–1243, 2023.

[CDL+12] Denis Cousineau, Damien Doligez, Leslie Lamport, Stephan Merz, Daniel Ricketts, and Hernán Vanzetto. TLA+ proofs. In *Proceedings of FM 2012*, volume 7436 of *Lecture Notes in Computer Science*, pages 147–154. Springer, 2012.

[CEFJ96] Edmund M. Clarke, Reinhard Enders, Thomas Filkorn, and Somesh Jha. Exploiting symmetry in temporal logic model checking. *Formal Methods in System Design*, 9(1–2):77–104, 1996.

[CGJ97] Edmund M. Clarke, Orna Grumberg, and Somesh Jha. Verifying parameterized networks. *ACM Transactions on Programming Languages and Systems*, 19(5):726–750, 1997.

[CGK+12] Sylvain Conchon, Amit Goel, Sava Krstić, Alain Mebsout, and Fatiha Zaïdi. Cubicle: A parallel SMT-based model checker for parameterized systems. In *Proceedings of CAV 2012*, volume 7358 of *Lecture Notes in Computer Science*, pages 718–724. Springer, 2012.

[CGK+13] Sylvain Conchon, Amit Goel, Sava Krstić, Alain Mebsout, and Fatiha Zaïdi. Invariants for finite instances and beyond. In *Proceedings of FMCAD 2013*, pages 61–68. IEEE, 2013.

[CHLR17] Yu-Fang Chen, Chih-Duo Hong, Anthony W. Lin, and Philipp Rümmer. Learning to prove safety over parameterised concurrent systems. In *Proceedings of FMCAD 2017*, pages 76–83. IEEE, 2017.

[CMP04] Ching-Tsun Chou, Phanindra K. Mannava, and Seungjoon Park. A simple method for parameterized verification of cache coherence protocols. In *Proceedings of*

FMCAD 2004, volume 3312 of *Lecture Notes in Computer Science*, pages 382–398. Springer, 2004.

[CTTV04] Edmund M. Clarke, Muralidhar Talupur, Tayssir Touili, and Helmut Veith. Verification by network decomposition. In *Proceedings of CONCUR 2004*, volume 3170 of *Lecture Notes in Computer Science*, pages 276–291. Springer, 2004.

[CTV06] Edmund M. Clarke, Muralidhar Talupur, and Helmut Veith. Environment abstraction for parameterized verification. In *Proceedings of VMCAI 2006*, volume 3855 of *Lecture Notes in Computer Science*, pages 126–141. Springer, 2006.

[CTV08] Edmund M. Clarke, Muralidhar Talupur, and Helmut Veith. Proving Ptolemy right: The environment abstraction framework for model checking concurrent systems. In *Proceedings of TACAS 2008*, volume 4963 of *Lecture Notes in Computer Science*, pages 33–47. Springer, 2008.

[Del00] Giorgio Delzanno. Automatic verification of parameterized cache coherence protocols. In *Proceedings of CAV 2000*, volume 1855 of *Lecture Notes in Computer Science*, pages 53–68. Springer, 2000.

[DLS01] Dennis Dams, Yassine Lakhnech, and Martin Steffen. Iterating transducers. In *Proceedings of CAV 2001*, volume 2102 of *Lecture Notes in Computer Science*, pages 286–297. Springer, 2001.

[DSZ10] Giorgio Delzanno, Arnaud Sangnier, and Gianluigi Zavattaro. Parameterized verification of ad hoc networks. In *Proceedings of CONCUR 2010*, volume 6269 of *Lecture Notes in Computer Science*, pages 313–327. Springer, 2010.

[EFM99] Javier Esparza, Alain Finkel, and Richard Mayr. On the verification of broadcast protocols. In *Proceedings of LICS 1999*, pages 352–359. IEEE Computer Society, 1999.

[EGLM17] Javier Esparza, Pierre Ganty, Jérôme Leroux, and Rupak Majumdar. Verification of population protocols. *Acta Informatica*, 54(2):191–215, 2017.

[EGM16] Javier Esparza, Pierre Ganty, and Rupak Majumdar. Parameterized verification of asynchronous shared-memory systems. *Journal of the ACM*, 63(1):1–48, 2016.

[EK00] E. Allen Emerson and Vineet Kahlon. Reducing model checking of the many to the few. In *Proceedings of CADE 2000*, volume 1831 of *Lecture Notes in Artificial Intelligence*, pages 236–254. Springer, 2000.

[EK03] E. Allen Emerson and Vineet Kahlon. Exact and efficient verification of parameterized cache coherence protocols. In *Proceedings of CHARME 2003*, volume 2860 of *Lecture Notes in Computer Science*, pages 247–262. Springer, 2003.

[EMB11] Niklas Eén, Alan Mishchenko, and Robert K. Brayton. Efficient implementation of property directed reachability. In *Proceedings of FMCAD 2011*, pages 125–134. FMCAD Inc., 2011.

[EN95] E. Allen Emerson and Kedar S. Namjoshi. Reasoning about rings. In *Proceedings of POPL 1995*, pages 85–94. ACM, 1995.

[ES96] E. Allen Emerson and A. Prasad Sistla. Symmetry and model checking. *Formal Methods in System Design*, 9(1–2):105–131, 1996.

[Esp98] Javier Esparza. Decidability and complexity of Petri net problems: An introduction. In *Lectures on Petri Nets I: Basic Models*, volume 1491 of *Lecture Notes in Computer Science*, pages 374–428. Springer, 1998.

[Esp14] Javier Esparza. Keeping a crowd safe: On the complexity of parameterized verification. In *Proceedings of STACS 2014*, volume 25 of *Leibniz International Proceedings in Informatics*, pages 1–10. Schloss Dagstuhl, 2014.

[FKP15] Azadeh Farzan, Zachary Kincaid, and Andreas Podelski. Proof spaces for unbounded parallelism. In *Proceedings of POPL 2015*, pages 407–420. ACM, 2015.

[FQ03] Cormac Flanagan and Shaz Qadeer. Thread-modular model checking. In *Proceedings of SPIN 2003*, volume 2648 of *Lecture Notes in Computer Science*, pages 213–224. Springer, 2003.

[FS01] Alain Finkel and Philippe Schnoebelen. Well-structured transition systems everywhere! *Theoretical Computer Science*, 256(1–2):63–92, 2001.

[GR10a] Silvio Ghilardi and Silvio Ranise. Backward reachability of array-based systems by SMT solving: Termination and invariant synthesis. *Logical Methods in Computer Science*, 6(4), 2010.

[GR10b] Silvio Ghilardi and Silvio Ranise. MCMT: A model checker modulo theories. In *Proceedings of IJCAR 2010*, volume 6173 of *Lecture Notes in Computer Science*, pages 22–29. Springer, 2010.

[Gro96] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of STOC 1996*, pages 212–219. ACM, 1996.

[GS92] Steven M. German and A. Prasad Sistla. Reasoning about systems with many processes. *Journal of the ACM*, 39(3):675–735, 1992.

[GS97] Viktor Gyuris and A. Prasad Sistla. On-the-fly model checking under fairness that exploits symmetry. In *Proceedings of CAV 1997*, volume 1254 of *Lecture Notes in Computer Science*, pages 232–243. Springer, 1997.

[GSM16] Arie Gurfinkel, Sharon Shoham, and Yuri Meshman. SMT-based verification of parameterized systems. In *Proceedings of FSE 2016*, pages 338–348. ACM, 2016.

[Hig52] Graham Higman. Ordering by divisibility in abstract algebras. *Proceedings of the London Mathematical Society*, s3-2(1):326–336, 1952.

[HMP17] Jochen Hoenicke, Rupak Majumdar, and Andreas Podelski. Thread modularity at many levels: A pearl in compositional verification. In *Proceedings of POPL 2017*, pages 473–485. ACM, 2017.

[HRH+21] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–29, 2021.

[HV05] Peter Habermehl and Tomáš Vojnar. Regular model checking using inference of regular languages. *Electronic Notes in Theoretical Computer Science*, 138(3):21–36, 2005.

[ID96] C. Norris Ip and David L. Dill. Better verification through symmetry. *Formal Methods in System Design*, 9(1–2):41–75, 1996.

[JKS+13] Annu John, Igor Konnov, Ulrich Schmid, Helmut Veith, and Josef Widder. Parameterized model checking of fault-tolerant distributed algorithms by abstraction. In *Proceedings of FMCAD 2013*, pages 201–209. IEEE, 2013.

[KBI+17] Aleksandr Karbyshev, Nikolaj Bjørner, Shachar Itzhaky, Noam Rinetzky, and Sharon Shoham. Property-directed inference of universal invariants or proving their absence. *Journal of the ACM*, 64(1):1–33, 2017.

[KGMC22] Shankaranarayanan Krishna, Adwait Godbole, Roland Meyer, and Soham Chakraborty. Parameterized verification under release acquire is PSPACE-complete. In *Proceedings of PODC 2022*, pages 482–492. ACM, 2022.

[KKW10] Alexander Kaiser, Daniel Kroening, and Thomas Wahl. Dynamic cutoff detection in parameterized concurrent programs. In *Proceedings of CAV 2010*, volume 6174 of *Lecture Notes in Computer Science*, pages 645–659. Springer, 2010.

[KLVW17] Igor Konnov, Marijana Lazić, Helmut Veith, and Josef Widder. A short counterexample property for safety and liveness verification of fault-tolerant distributed algorithms. In *Proceedings of POPL 2017*, pages 719–734. ACM, 2017.

[KM89] Robert P. Kurshan and Kenneth L. McMillan. A structural induction theorem for processes. In *Proceedings of PODC 1989*, pages 239–247. ACM, 1989.

[KMM+01] Yonit Kesten, Oded Maler, Monica Marcus, Amir Pnueli, and Elad Shahar. Symbolic model checking with rich assertional languages. *Theoretical Computer Science*, 256(1–2):93–112, 2001.

[KPIA20] Jason R. Koenig, Oded Padon, Neil Immerman, and Alex Aiken. First-order quantified separators. In *Proceedings of PLDI 2020*, pages 703–717. ACM, 2020.

[KPSZ02] Yonit Kesten, Amir Pnueli, Elad Shahar, and Lenore Zuck. Network invariants in action. In *Proceedings of CONCUR 2002*, volume 2421 of *Lecture Notes in Computer Science*, pages 101–115. Springer, 2002.

[KVW17] Igor Konnov, Helmut Veith, and Josef Widder. On the completeness of bounded model checking for threshold-based distributed algorithms: Reachability. *Information and Computation*, 252:95–109, 2017.

[Lam79] Leslie Lamport. How to make a multiprocessor computer that correctly executes multiprocess programs. *IEEE Transactions on Computers*, C-28(9):690–691, 1979.

[Lam94] Leslie Lamport. The temporal logic of actions. *ACM Transactions on Programming Languages and Systems*, 16(3):872–923, 1994.

[Lam02] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[LB07] Shuvendu K. Lahiri and Randal E. Bryant. Predicate abstraction with indexed predicates. *ACM Transactions on Computational Logic*, 9(1), 2007.

[LHR97] David Lesens, Nicolas Halbwachs, and Pascal Raymond. Automatic verification of parameterized linear networks of processes. In *Proceedings of POPL 1997*, pages 346–357. ACM, 1997.

[LR16] Anthony W. Lin and Philipp Rümmer. Liveness of randomised parameterised systems under arbitrary schedulers. In *Proceedings of CAV 2016*, volume 9779 of *Lecture Notes in Computer Science*, pages 112–133. Springer, 2016.

[Mar94] Alberto Marcone. Foundations of BQO theory. *Transactions of the American Mathematical Society*, 345(2):641–660, 1994.

[McM01] Kenneth L. McMillan. Parameterized verification of the FLASH cache coherence protocol by compositional model checking. In *Proceedings of CHARME 2001*, volume 2144 of *Lecture Notes in Computer Science*, pages 179–195. Springer, 2001.

[MGJ+19] Haojun Ma, Aman Goel, Jean-Baptiste Jeannin, Manos Kapritsos, Baris Kasikci, and Karem A. Sakallah. I4: Incremental inference of inductive invariants for verification of distributed protocols. In *Proceedings of SOSP 2019*, pages 370–384. ACM, 2019.

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 10th anniversary edition, 2010.

[NJ13] Daniel Neider and Nils Jansen. Regular model checking using solver technologies and automata learning. In *Proceedings of NFM 2013*, volume 7871 of *Lecture Notes in Computer Science*, pages 16–31. Springer, 2013.

[NRZ+15] Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon web services uses formal methods. *Communications of the ACM*, 58(4):66–73, 2015.

[OG76] Susan Owicki and David Gries. An axiomatic proof technique for parallel programs I. *Acta Informatica*, 6(4):319–340, 1976.

[OSS09] Scott Owens, Susmit Sarkar, and Peter Sewell. A better x86 memory model: x86-TSO. In *Proceedings of TPHOLs 2009*, volume 5674 of *Lecture Notes in Computer Science*, pages 391–407. Springer, 2009.

[PLSS17] Oded Padon, Giuliano Losa, Mooly Sagiv, and Sharon Shoham. Paxos made EPR: Decidable reasoning about distributed protocols. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA):1–31, 2017.

[PMP+16] Oded Padon, Kenneth L. McMillan, Aurojit Panda, Mooly Sagiv, and Sharon Shoham. Ivy: Safety verification by interactive generalization. In *Proceedings of PLDI 2016*, pages 614–630. ACM, 2016.

[PRZ01] Amir Pnueli, Sitvanit Ruah, and Lenore Zuck. Automatic deductive verification with invisible invariants. In *Proceedings of TACAS 2001*, volume 2031 of *Lecture Notes in Computer Science*, pages 82–97. Springer, 2001.

[PXZ02] Amir Pnueli, Jessie Xu, and Lenore Zuck. Liveness with $(0, 1, \infty)$-counter abstraction. In *Proceedings of CAV 2002*, volume 2404 of *Lecture Notes in Computer Science*, pages 107–122. Springer, 2002.

[Rac78] Charles Rackoff. The covering and boundedness problems for vector addition systems. *Theoretical Computer Science*, 6(2):223–231, 1978.

[RMV22] Azalea Raad, Luc Maranget, and Viktor Vafeiadis. Extending Intel-x86 consistency and persistency: Formalising the semantics of Intel-x86 memory types and non-temporal stores. *Proceedings of the ACM on Programming Languages*, 6(POPL):1–31, 2022.

[SG99] A. Prasad Sistla and Viktor Gyuris. Parameterized verification of linear networks using automata as invariants. *Formal Aspects of Computing*, 11(4):402–425, 1999.

[Sho97] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[SSO+10] Peter Sewell, Susmit Sarkar, Scott Owens, Francesco Zappa Nardelli, and Magnus O. Myreen. x86-TSO: A rigorous and usable programmer's model for x86 multiprocessors. *Communications of the ACM*, 53(7):89–97, 2010.

[Suz88] Ichiro Suzuki. Proving properties of a ring of finite-state machines. *Information Processing Letters*, 28(4):213–214, 1988.

[TT08] Muralidhar Talupur and Mark R. Tuttle. Going with the flow: Parameterized verification using message flows. In *Proceedings of FMCAD 2008*, pages 1–8. IEEE, 2008.

[WB98] Pierre Wolper and Bernard Boigelot. Verifying systems with infinite but regular state spaces. In *Proceedings of CAV 1998*, volume 1427 of *Lecture Notes in Computer Science*, pages 88–97. Springer, 1998.

[WL89] Pierre Wolper and Vinciane Lovinfosse. Verifying properties of large sets of processes with network invariants. In *Proceedings of the International Workshop on Automatic Verification Methods for Finite State Systems*, volume 407 of *Lecture Notes in Computer Science*, pages 68–80. Springer, 1989.

[Yin11] Mingsheng Ying. Floyd–Hoare logic for quantum programs. *ACM Transactions on Programming Languages and Systems*, 33(6):1–49, 2011.

[ZP04] Lenore Zuck and Amir Pnueli. Model checking and abstraction to the aid of parameterized systems (a survey). *Computer Languages, Systems & Structures*, 30(3–4):139–169, 2004.